

ZLC FPGA Pulse Streamer 手册

Artix-7 35T 运行时边沿流送器 / N-slot 仿射扫描 / VIO 契约

RTL、扫描引擎、模拟总线 DAC、编译上传流程、资源预算与 v3 路线图

2026-06-06

Contents

1	写给新读者	1
1.1	这块硬件是什么	1
1.2	文件	1
2	运行时边沿表核	2
2.1	一行边沿的含义	2
2.2	为什么读路径很简单	2
2.3	时钟与对齐	2
3	N-slot 仿射扫描引擎	3
3.1	核心公式	3
3.2	一个具体数字例子	3
3.3	扫描点的无缝迭代	4
3.4	编译器的安全边界	4
3.5	repeat 是元数据	4
4	VIO probe 契约	5
4.1	打包后的宽度	5
4.2	完整 probe 表	5
5	模拟总线 DAC 引擎	7
5.1	为什么单独一张表	7
5.2	三种模式	7
5.3	无缝扫描 DAC 值 (value_select)	7
5.4	DAC 值 + duration + delay 同时扫 (仿射总线 tick)	8
6	主机编译与上传流程	9
6.1	从 PulseTableState 到 RuntimeSequenceProgram	9
6.2	prepare / fire	9
6.3	一行边沿是怎么写进 FPGA 的	9
6.4	差分上传	10
6.5	调试清单	10
6.6	命令行生成 HDL	10

7	35T 资源预算与收缩	11
7.1	默认 profile	11
7.2	怎么收缩	11
8	Build 与 program 工作流	13
8.1	batch 入口	13
8.2	启动 server	13
9	运行时边沿表核 (深入)	14
9.1	模块参数与整体形状	14
9.2	边沿行格式: 一行到底装了什么	15
9.2.1	三块边沿表存储的精确布局	15
9.2.2	有效 tick 的计算: 仿射修正	16
9.2.3	逐位算一遍: 一个真实的边沿行	16
9.3	为什么用分布式 RAM (LUTRAM) 而不是 BRAM	16
9.4	读通路 + 影子元数据	17
9.4.1	运行期只读当前一行	17
9.4.2	为什么要影子寄存器	17
9.5	时钟与对齐	18
9.6	同步器与边沿检测: 让 VIO 拨一下只写一次	18
9.7	上传契约: 六步把一张表灌进 FPGA	19
9.7.1	扫描点表的比特布局	19
9.7.2	一个完整上传的数值小例	19
10	N-slot 仿射扫描引擎 (深入)	21
10.1	为什么是“仿射 + 流式扫描点表”而不是 run-length 重写	21
10.2	仿射公式与 zlc_effective_tick 内部实现	22
10.2.1	参数与位宽	22
10.2.2	打包约定 (little-endian)	22
10.2.3	function 主体	23
10.3	流式扫描点表与无缝迭代 FSM	23
10.3.1	两份上传: 模板 + 扫描点表	23
10.3.2	无缝遍历的关键寄存器	24
10.3.3	无缝切点的 FSM 逻辑	24
10.4	一个完整的数值例子: 扫 duration s0 三个点	25
10.5	host 端安全边界 (编译期拒绝)	26
10.6	repeat 是元数据, 不是展开的行	26
11	模拟总线 DAC 引擎、value_select 与仿射总线 tick (深入)	27
11.1	为什么需要一张独立的总线表	27
11.2	段内存的逐字段布局	27
11.3	三种模式与 DDA 斜坡	28
11.4	value_select: DAC 值扫描路径与 slot_vec/NBA 时序坑	29
11.5	仿射总线 tick: zlc_bus_seg_start 与同时扫 DAC 值 + duration + delay	30

11.6	主机到 RTL 的数据流	31
12	VIO probe 契约与上传流程 (深入)	33
12.1	为什么用 VIO 而不是 AXI	33
12.2	顶层例化与 vio_0 核	33
12.3	完整 30 条 probe_out 表	34
12.4	为什么要打包宽探针	35
12.4.1	打包位序: _pack_signed	35
12.4.2	分布式 RAM 单读口的影响	36
12.5	契约测试: .v 与 Python 生成器对齐	36
12.6	上传流程: prepare (_prepare_tcl)	36
12.7	fire: 只认同步上升沿	37
12.8	差分上传 (differential upload)	37
13	JTAG-to-AXI 边沿表加载器 (当前架构)	39
13.1	为什么放弃 per-channel run-length	39
13.2	控制路径: 片上加载器 (替代 VIO)	39
13.3	各文件.	39
13.4	当前容量与待办.	40
13.5	构建 / 运行	40
14	Architecture-D: BRAM 表 + 预取 (2048 edges + 4096 points)	41
14.1	为什么需要 D	41
14.2	depth-1 预取与最小边沿间距	41
14.3	容量与参数化 (随 XDC + FPGA 派生)	41
14.4	各文件.	42
14.5	状态与 bring-up	42

1

写给新读者

1.1 这块硬件是什么

pulse-streamer 是一个**固定 bitstream + 运行时表**的脉冲序列器。烧一次 bitstream 就得到时钟状态机、address-switch 输出引脚和 Vivado VIO 控制 probe；它**不**把某个具体实验脉冲写死进 Verilog。之后每次 On Pulse 都把当前的 PulseTableState/PulseSequence 编译成一张紧凑的表，通过已经建好的 VIO probe 写进 FPGA RAM。

目标器件

本设计面向 **Xilinx Artix-7 35T (xc7a35tfgg484-2)**，FGG484 封装。这是一颗 LUT 资源有限的小器件，所以所有容量参数都按它来权衡。

近似资源类别：

xc7a35tfgg484-2

logic cells: 33,280	CLB LUTs: 20,800	CLB FFs: 41,600
BRAM: 1,800 Kb (50 x 36 Kb)	DSP: 90	

1.2 文件

fpga/pulse_streamer/zlc_pulse_streamer.v 运行时边沿表 pulse-streamer 核。由 NUM_SLOTS、CHANNEL_COUNT、MAX_EDGES、MAX_SCAN_POINTS、COEFF_FRAC_BITS 等参数化。

fpga/pulse_streamer/zlc_pulse_streamer_top_address_switch.v address-switch 顶层封装与 VIO 契约。NUM_SLOTS=4，62 路 channel + 4x10b DAC 总线。它暴露原 XDC 端口名，并映射到核的 out[0..61] mask bit。

Zou_lab_control/neutral_atom/devices/fpga_pulse_streamer.py Python HDL 生成器、宽度/容量校验器，以及 Vivado Tcl。

2

运行时边沿表核

2.1 一行边沿的含义

核里保存一张边沿表：

每行

tick[i]: 该边沿的整数 FPGA 时钟 tick
mask[i]: 该 tick 上完整的输出状态 (CHANNEL_COUNT bit)

bit 0 对应 `channels[0]`, bit 1 对应 `channels[1]`。多条 channel 在同一物理时刻变化, 合并成一条带完整 mask 的边沿。一行的含义是: **在这个绝对 FPGA tick, 把所有输出切换成这个 mask**——它不是逐 channel 命令列表。

2.2 为什么读路径很简单

shot 运行期间, 状态机只读**当前** `edge_index` 行, 再加上在上传时就锁存好的 first-edge、loop-start、final-tick 元数据。输出寄存器直接是 `out = state_mask`。这样即使大表用 distributed RAM (异步单端口读), 也不会有多读口 RAM 压力, 同时仍支持 repeat bracket 和差分上传。

2.3 时钟与对齐

真实硬件默认 50 MHz, 一个 tick 是 20 ns。Python 在上传前校验所有 period 持续时间、channel 延时、扫描点都对齐到这个 tick 网格 (`1e9 / clock_hz`)。32 位 tick 计数器在 50 MHz 下覆盖约 85.9 秒; 边沿量化误差至多半个 20 ns tick。同步后的 VIO start 路径会引入一个固定的小启动延迟, 但那是常数偏移, 不是比例因子; 脉冲宽度和延时由 tick 计数决定。

3

N-slot 仿射扫描引擎

3.1 核心公式

这是当前的扫描机制，取代了旧的 x/y 二变量仿射引擎——**已经没有 x/y** 。每行边沿存一个 base tick 加 `NUM_SLOTS` 个定点系数；运行时 FPGA 对每个扫描点计算：

$$\text{effective_tick} = \text{base_tick} + \left(\sum_j \text{coeff}_j \cdot \text{slot}_j \right) \gg \text{COEFF_FRAC_BITS} \quad (3.1)$$

其中 `COEFF_FRAC_BITS=8` (定点小数位)，`NUM_SLOTS=2` 时正好退化成旧的 (x, y) 仿射引擎。`NUM_SLOTS` 在 bitstream 里固定 (这里是 4)，按需要扫描的字段数量取的小整数。

3.2 一个具体数字例子

假设要把第 1 段 (成像) 的持续时间从 **1 us** 扫到 **5 us**，时钟 50 MHz (1 tick = 20 ns)。绑定后这个时长成为 slot `s0`。所有在成像段**之后**发生的边沿，其 base tick 都带一个对 `s0` 的系数 `coeff = 256` (即 $1 \ll \text{COEFF_FRAC_BITS}$ ，表示“1 个 tick 的 slot 值贡献 1 个 tick”)。

逐扫描点求值

```
slot s0 取值 (tick) :   50, 100, 150, 200, 250           # 1us..5us / 20ns
某条后续边沿 base_tick = 1000, coeff = 256
effective_tick = 1000 + (256 * s0) >> 8
                  = 1000 + s0
                  -> 1050, 1100, 1150, 1200, 1250       # 每点自动平移
```

主机上传的是一张边沿模板 (base_tick + 系数) 和一张扫描点表 (每行的 `s0..` 值)。FPGA 对每个扫描点用上式重算所有边沿，无需为 5 个点存 5 份表。扫 1000 个点也只是一张 1000 行的扫描点表，模板不变长。

3.3 扫描点的无缝迭代

主机上传一张边沿模板加一张流式扫描点表 (`scan_value_mem`, 每行 `NUM_SLOTS` 个 tick 值)。FPGA 内部用 `scan_point_index` 顺序遍历扫描点: 跑完当前点的整张模板后, 若还有下一个点 (`scan_point_index + 1 < active_scan_count`), 就自增索引、用新 slot 值重新求值 `effective_tick`, 无缝进入下一点, 每点对外触发一次相机。

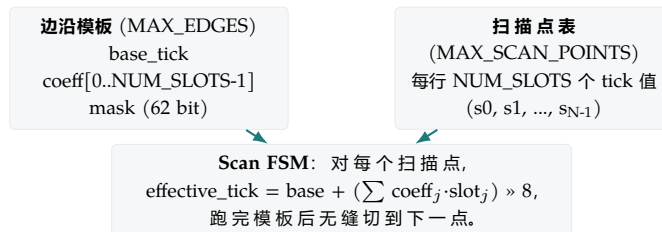


Figure 3.1: 一张模板 + 一张扫描点表。扫描点数不会让模板变长——边沿限制的是模板复杂度。

3.4 编译器的安全边界

主机编译器 (`sequencer.py` 的 `compile_pulse_table_scan_runtime_program`) 只接受 slot 的仿射时间表达式。它会拒绝: 非仿射的扫描时序; 以及在不同扫描点上边沿顺序会交换的模板 (一个固定 edge row 顺序无法安全描述互相矛盾的 timing)。遇到这两种情况应拆成多个 chunk, 或每个点单独 prepare/fire。

3.5 repeat 是元数据

repeat 不是展开的行。整表无限循环用 `repeat_forever=1` 上传一次。有限内层 bracket 用 `loop_start_addr`、`loop_end_tick`、`loop_count` (loop end 也带一组 `loop_end_coeffs` 仿射系数)。running/done 由 `prog_count` 和当前索引驱动。

4

VIO probe 契约

4.1 打包后的宽度

NUM_SLOTS=4 时, 三个仿射相关 probe 被打包:

Packed widths

```
prog_tick_coeffs : NUM_SLOTS*COEFF_WIDTH = 4*16 = 64 bit
scan_prog_values : NUM_SLOTS*TICK_WIDTH  = 4*32 = 128 bit
loop_end_coeffs  : NUM_SLOTS*COEFF_WIDTH = 4*16 = 64 bit
```

4.2 完整 probe 表

顶层共 30 个 probe_out 和 2 个 probe_in:

VIO probes (NUM-SLOTS=4)

probe_out0	zlc_reset	width 1	
probe_out1	zlc_start	width 1	
probe_out2	zlc_prog_we	width 1	
probe_out3	zlc_prog_addr	width 10	
probe_out4	zlc_prog_tick	width 32	
probe_out5	zlc_prog_mask	width 62	
probe_out6	zlc_prog_count	width 11	
probe_out7	zlc_repeat_forever	width 1	
probe_out8	zlc_loop_start_addr	width 10	
probe_out9	zlc_loop_end_tick	width 32	
probe_out10	zlc_loop_count	width 32	
probe_out11	zlc_prog_tick_coeffs	width 64	(4*16)
probe_out12	zlc_scan_enable	width 1	
probe_out13	zlc_scan_prog_we	width 1	
probe_out14	zlc_scan_prog_addr	width 10	
probe_out15	zlc_scan_prog_values	width 128	(4*32)
probe_out16	zlc_scan_count	width 11	
probe_out17	zlc_loop_end_coeffs	width 64	(4*16)

```
probe_out18 zlc_bus_prog_we          width 1
probe_out19 zlc_bus_prog_bus         width 2
probe_out20 zlc_bus_prog_addr        width 6
probe_out21 zlc_bus_prog_start_tick  width 32
probe_out22 zlc_bus_prog_stop_tick   width 32
probe_out23 zlc_bus_prog_start_value width 10
probe_out24 zlc_bus_prog_stop_value  width 10
probe_out25 zlc_bus_prog_mode        width 2
probe_out26 zlc_bus_counts           width 28
probe_out27 zlc_bus_prog_value_select width 3  (0=literal, j+1=scan
↪ slot j)
probe_out28 zlc_bus_prog_start_tick_coeffs width 64 (4*16, affine bus
↪ tick)
probe_out29 zlc_bus_prog_stop_tick_coeffs width 64 (4*16, affine bus
↪ tick)
probe_in0   zlc_running              width 1
probe_in1   zlc_done                 width 1
```

这张表必须和顶层 `zlc_pulse_streamer_top_address_switch.v` 里的 `vio_0` 实例保持一致。一个 Python 契约测试会断言生成的 VIO 宽度与 Python 生成器匹配。

5

模拟总线 DAC 引擎

5.1 为什么单独一张表

address-switch XDC 里有四组 10-bit TTL 总线: `da_dipole`、`da_bias_y`、`da_bias_x`、`da_bias_z` (硬件顺序如此, 每组对应核 `bus_out` 的 10 个 bit)。如果把一条长 ramp 的每个台阶都展开成普通 `prog_mask` 边沿, 会很快填满边沿表。所以 DAC 走**独立的总线段表**, 普通 TTL 继续走 `tick/mask`。

一行总线段

```
bus_id, start_tick, stop_tick, start_value, stop_value, mode, value_select
```

`value_select` 是为**扫描 DAC 值**加的字段: 0 表示用上面的字面 `start_value/stop_value`; `j+1` 表示运行时从扫描 slot `j` 取 DAC 码 (见下一节)。

5.2 三种模式

每条总线一个小引擎, 每个 FPGA tick 检查当前段:

edge `start_tick = stop_tick`, 跑到该 tick 直接换值。

ramp 用 `start_tick/stop_tick` 和 `start_value/stop_value`, 内部 DDA accumulator 每个 tick 算出当前阶梯, 到 `stop_tick` 时 snap 到 `stop_value`。一条长 ramp 只占**一个**段。

hold 用“没有新段”表示, 沿用当前值或正在进行的 ramp。

每条总线最多 64 个段; `bus_counts` 记录每条总线的有效段数, 未使用的总线 count 为 0, 所以旧 RAM 内容不会执行。

5.3 无缝扫描 DAC 值 (`value_select`)

DAC 值可以**无缝**硬件扫描——和数字时序扫描共用同一张扫描点表, 不需要每点重新 prepare。机制是给每个总线段加一个 `value_select` 选择位:

`value_select = 0` 段用上传的字面 `start_value/stop_value` (老行为, 完全不变)。

value_select = j+1 段在被应用时, 从**当前扫描点**的 slot **j** 读低 **BUS_WIDTH**(=10) 位作为 DAC 码。扫描点前进时, slot 值自动更新, DAC 输出随之逐点切换。

关键时序: 段应用任务 (`zlc_bus_apply_segment`) 的 slot 向量是**显式参数**传入的, 而不是直接读 `slot_active` 寄存器。这是因为在扫描点边界那一拍, `slot_active` 的非阻塞赋值还没生效; 若直接读会拿到上一个点的旧值。改为把“本扫描点应当生效的 slot 向量”(启动时是 `scan_first_values`, 前进时是 `scan_value_mem[next]`) 当参数传下去, 进入新点的第一拍就是正确值。

主机侧: DAC 扫描 slot 的列在扫描点表里存**原始 10 位 DAC 码** (不做 ns→tick 换算), 且其仿射系数恒为 0, 所以它从不进入 `effective_tick` 公式——只被总线引擎读取。

DAC 扫描的一个具体例子

```
# 扫 da_dipole 在第 1 段的电平: 码 0 -> 256 -> 768 -> 1023
scan_points (slot 列):  [0], [256], [768], [1023]    # 原始 10-bit 码
该段的总线段: bus=0, start_tick=5, mode=edge, value_select=1
# value_select=1 表示运行时取 slot 0 的低 10 位作 DAC 码。
扫描点 0: DAC = 0      扫描点 1: DAC = 256
扫描点 2: DAC = 768    扫描点 3: DAC = 1023
```

这正是仓库里 `test_dac_value_scan_behavioral_model_tracks_scanned_code` 用 RTL 总线引擎的 Python 等价模型验证过的行为: 每个扫描点的 DAC 输出等于该点的码。

5.4 DAC 值 + duration + delay 同时扫 (仿射总线 tick)

现在每个总线段的 start/stop tick 也带一组**仿射系数** (和数字边沿完全一样), FPGA 用同一个 `zlc_effective_tick` 求值:

$$\text{effective_bus_tick} = \text{base_tick} + \left(\sum_j \text{coeff}_j \cdot \text{slot}_j \right) \gg \text{COEFF_FRAC_BITS} \quad (5.1)$$

于是扫某个 duration 时, 它后面所有总线段的 effective tick 会和数字边沿**同步平移**; 而 **value_select** 让段的 DAC 值从槽读取。两者正交, 所以 **DAC 值 + duration + delay 可以任意组合同时扫**——这正是早期版本做不到、需要重新设计的地方。系数存在 `bus_start_tick_coeff_mem` / `bus_stop_tick_coeff_mem`, 经 probe_out28/29 上传; 求值入口是 `zlc_bus_seg_start` 和 `zlc_bus_apply_segment`。

当前唯一限制

模拟 ramp 的端点不能是被扫的 DAC 值 (用 edge 保持那个被扫的电平)。除此之外, DAC 值、duration、delay 的任意组合都能在一次上传里硬件无缝扫描。仓库里 `test_dac_plus_duration_scan_behavioral_model_value_and_timing` 用 RTL 总线引擎的 Python 等价模型验证: 扫 duration 时被扫的 DAC 码出现在**平移后**的 tick 上, 值与时序同时正确。

6

主机编译与上传流程

6.1 从 PulseTableState 到 RuntimeSequenceProgram

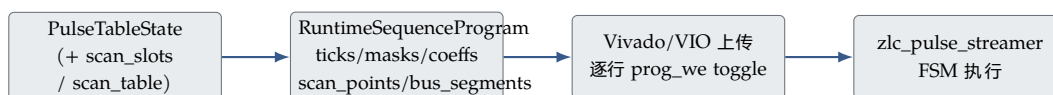


Figure 6.1: 编译 -> RuntimeSequenceProgram -> VIO 上传 -> FPGA 执行。

RuntimeSequenceProgram 的 schema 携带 `slot_count`、`slot_kinds`、`tick_slot_coeffs`、`loop_end_slot_coeffs`、`scan_points`、`scan_coeff_frac_bits`，加上 `ticks/masks/bus_segments`。

6.2 prepare / fire

prepare 时 backend 拉高 reset (输出寄存器强制为 0, 写侧使能)。逐行 stage `prog_addr`、`prog_tick`、`prog_tick_coeffs`、`prog_mask`，再 toggle `prog_we`；FPGA 在那次同步 toggle 上恰好写一次。扫描点经 `scan_prog_addr/scan_prog_values/scan_prog_we` 上传；总线段经 `bus_prog_*` 上传。然后释放 reset。**prepare 不启动**。fire 发一个显式 `zlc_start` 低-高-低脉冲；FPGA 只把同步上升沿当 start。此后 shot 内时序由 FPGA 时钟拥有。

6.3 一行边沿是怎么写进 FPGA 的

每行边沿不是一次写一个寄存器，而是**先 stage 一组 probe，再 toggle 一个写使能**。host 端 (`fpga_pulse_streamer.py` 的 `_prepare_tcl`) 对每行边沿做：

一行边沿的上传序列

```
stage prog_addr      = i          # 行号
stage prog_tick      = base_tick
stage prog_tick_coeffs = 打包的 NUM_SLOTS 个系数
stage prog_mask      = 62-bit 输出 mask
toggle prog_we              # 上升沿：FPGA 把上面这组写进第 i 行
```

扫描点经 `scan_prog_addr/scan_prog_values/scan_prog_we` 同样方式写；总线段经 `bus_prog_*` (含新增的 `bus_prog_value_select`) 写。因为是 toggle 触发, host 不需要精确计时——FPGA 在每次 `*_we` 跳变上恰好写一次。

6.4 差分上传

第一次成功 prepare 后,持久会话保留一份已上传表的 Python 副本。只要 channel order 和 clock 不变,后续 prepare 只重写改变的边沿行、改变的扫描点,以及 shadow-critical 行(第 0 行、`loop_start_index`、最后一行)。

6.5 调试清单

板子不出预期波形时,按这个顺序查:

1. `build_and_program.bat -check`: 不连板就能跑 HDL/VIO 宽度自查 + 容量预算,确认 Python 生成器和 `.v` 一致 (Python 契约测试也查这个)。
2. 确认烧进去的 `.bit/.ltx` 与 `run_server.bat -check-config` 指向的是同一套。
3. 示波器脉冲长度翻倍 → 还在按 100 MHz 编译; 确认 server/GUI/notebook 都是 50 MHz / 20 ns。
4. DAC 不动 → 确认该总线的 `bus_counts` 非 0、段的 `start_tick` 在表范围内; 扫 DAC 时确认 `value_select` 指向正确的 slot 列。
5. 扫描乱序 → 编译器会拒绝在不同扫描点交换边沿顺序的模板; 拆 chunk 或简化 delay/duration 表达式。

6.6 命令行生成 HDL

PowerShell

```
python -m Zou_lab_control.neutral_atom.devices.fpga_pulse_streamer
↪ generate_hdl `
  --output-dir D:\zlc_pulse_streamer_hdl `
  --channels trap cooling probe trig `
  --max-edges 1024 `
  --tick-width 32
```

7

35T 资源预算与收缩

7.1 默认 profile

Default profile

CHANNEL_COUNT	= 62	TICK_WIDTH	= 32
NUM_SLOTS	= 4	COEFF_WIDTH	= 16
MAX_EDGES	= 1024	COEFF_FRAC_BITS	= 8
MAX_SCAN_POINTS	= 1024	CLOCK_HZ	= 50 MHz
RESOURCE_TARGET	= 70%		

每行边沿约 32 (tick) + 62 (mask) + NUM_SLOTS*16 (coeff) bit (4 slot 时 158 bit)；每个扫描点 NUM_SLOTS*32 bit (4 slot 时 128 bit)。tick_mem、mask_mem、coeff_mem、scan_value_mem 当前都标了 ram_style = "distributed" (LUTRAM)，因为运行路径用简单的异步单行读。这对 35T 有限的 LUT 是直接压力。

具体预算 (设计估算, 非综合结果)

capacity_estimate (fpga_pulse_streamer.py) 按“每比特 LUTRAM 行 / 64 深”建模：

- 默认 4-slot (62 通道 / 1024 边沿 / 1024 扫描点)：约 5830 LUT $\approx$ 28% (共 20800 LUT)。
- 70% LUT 预算内：固定 1024 扫描点时约可到 max_edges $\approx$ 4560；固定 1024 边沿时约可到 max_scan_points $\approx$ 5389。
- slot 数翻倍(4 $\rightarrow$ 8)约升到 42.8%；slot=2 约 20.6%——coeff/scan-value 这两块 LUTRAM 与 NUM_SLOTS 成正比。
- 扫描点表 (1024 $\times$ 128 bit) 若改 ram_style 到 BRAM 约 4/50 个 36 Kb 块。

最终以 Vivado report_utilization 为准。

7.2 怎么收缩

按代价从低到高：

1. 先看序列能否用 repeat bracket、`repeat_forever`、符号化 slot 模板、或更小的 scan chunk 表达，而不是堆大量唯一边沿。
2. 降低 `MAX_EDGES` / `MAX_SCAN_POINTS` (build 前用环境变量)。
3. 若确实需要上万条唯一非重复边沿：把大表从 distributed LUTRAM 迁到 **BRAM** 友好的同步读流水线，并换更快的上传 transport (AXI、JTAG-to-AXI、UART/SPI、以太网、或 FIFO/BRAM 写口)。35T 有 50 个 36 Kb BRAM，足够承载远大于 1024 行的表。

PowerShell

```
$env:ZLC_PS_RESOURCE_TARGET_PCT = "70"  
$env:ZLC_PS_MAX_EDGES = "1024"  
$env:ZLC_PS_MAX_SCAN_POINTS = "1024"  
.\fpga\build_and_program.bat --check
```

Python/Tcl 打印的容量是 build 前的预算提示；最终证据是 Vivado `report_utilization` 和 `report_timing_summary`。

8

Build 与 program workflow

8.1 batch 入口

从仓库根目录（保持短路径，如 `D:\ZLC`，因为 Vivado 2019 debug core 对路径长度敏感）：

PowerShell

```
.\fpga\build_and_program.bat --help
.\fpga\build_and_program.bat --check      # 不连板 HDL/VIO 宽度 + 容量自查
.\fpga\build_and_program.bat              # build + program
.\fpga\build_and_program.bat --build-only  # 只 build
.\fpga\build_and_program.bat --program-only # 只 program 现有 bit/LTX
.\fpga\build_and_program.bat --diagnose    # 列 Vivado hw target/device
```

默认工程 `fpga\build \address_switch`；产物在其 `address_switch.runs\impl_1` 下的 `.bit/.ltx`。打印的 `ZLC build root / ZLC project dir` 是 `.xpr/.bit/.ltx` 的真相来源。默认 XDC 是历史 `address-switch` 引脚图；换板用 `ZLC_PS_XDC`。Vivado 发现顺序：`ZLC_PS_VIVADO_BIN` -> 常见安装路径 -> `PATH`。

8.2 启动 server

PowerShell

```
.\fpga\run_server.bat --check-config  # 打印
↪ project/bit/ltx/XDC/channel/trigger/clock/容量
.\fpga\run_server.bat                  # host 0.0.0.0, port 18861,
↪ vivado-session 后端
```

9

运行时边沿表核（深入）

本章深入剖析 `fpga/pulse/_streamer/zlc/_pulse/_streamer.v` 中的 **运行时边沿表核**——即 `module zlc_pulse_streamer` 在真正发脉冲时所依赖的存储结构、读取通路、时钟对齐策略与上位机上传契约。目标读者是要亲手改 RTL 的硬件工程师：读完本章，你应当能解释每一条信号、每一块存储、每一个 task 为什么长成现在这个样子，能算出任意一行边沿的精确比特布局，并能在不破坏现有契约的前提下扩展它。

本核心服务于一块 **Artix-7 35T (xc7a35tfgg484-2, 约 20.8k LUT)** 的小器件，且仓库里**没有 Verilog 仿真器**。这两个约束贯穿全章：它们直接决定了“为什么用分布式 RAM 而不是 BRAM”“为什么读通路只读当前一行”“为什么用影子寄存器把元数据搬到上传期”。

9.1 模块参数与整体形状

`module zlc_pulse_streamer` 完全参数化。下面列出与边沿表核相关的全部参数及其默认值，后续所有比特布局都从这些常量推导出来。

CHANNEL_COUNT = 62 数字输出通道数。输出寄存器、mask、每一行边沿的掩码宽度都是 62 位。bit 0 对应 `channels[0]`。

EDGE_ADDR_WIDTH = 10 边沿表地址宽度，推出 **MAX_EDGES = 1024**。即一次拍摄最多 1024 条边沿。

TICK_WIDTH = 32 绝对 tick 计数宽度。32 位计数器在 50 MHz (20 ns/tick) 下覆盖约 $2^{32} \times 20\text{ns} \approx 85.9$ 秒。

SCAN_ADDR_WIDTH = 10 扫描点表地址宽度，推出 **MAX_SCAN_POINTS = 1024**。即一次扫描最多 1024 个扫描点。

NUM_SLOTS = 4 命名扫描 slot 数。一次拍摄最多可同时绑定 4 个被扫字段（duration / delay / DAC 值）。

COEFF_WIDTH = 16 每个 slot 的有符号仿射系数宽度（含符号位）。

COEFF_FRAC_BITS = 8 仿射系数的定点小数位数。 `effective_tick = base + (sum_j coeff_j * slot_j) » 8`。

BUS_COUNT = 4 模拟总线 (DAC) 数。

BUS_WIDTH = 10 每条 DAC 总线位宽 (10 位码, 0–1023)。

BUS_SEG_ADDR_WIDTH = 6 总线段地址宽度 (每条总线最多 64 段)。

由这些参数派生出几个关键的“组合宽度”常量，它们决定了存储块每行的位数：

COEFF_BITS = NUM_SLOTS × COEFF_WIDTH = 4 × 16 = 64 一行边沿的系数打包宽度。把 4 个有符号 16 位系数横向拼成一个 64 位字存进 LUTRAM。

SLOT_BITS = NUM_SLOTS × TICK_WIDTH = 4 × 32 = 128 一个扫描点的打包宽度。每个 slot 在扫描点表里占满 32 位 (即一个完整 tick 宽度)，4 个 slot 拼成 128 位。

一句话概括核心

边沿表核 = “1024 行 × (32 位绝对基准 tick + 64 位打包系数 + 62 位输出掩码)” 的分布式 RAM，配一张“1024 点 × 128 位打包 slot 值”的扫描点表，外加一组在上传时锁存、运行时只读的影子元数据寄存器。运行期 FSM 每拍只读**当前一行**，因此整张表无需多读口。

9.2 边沿行格式：一行到底装了什么

理解整个核心的钥匙是**一行边沿的语义**。一行边沿 = 一个绝对基准 tick + 每个 slot 一个有符号系数 + 一整张 62 位输出掩码。它的含义是：

“到了这个 (经仿射修正后的有效) tick，把全部 62 个输出寄存器一次性设成这张掩码。”

注意这是**绝对掩码赋值**，不是“翻转某几位”。`assign out = state_mask`——输出寄存器本身就是当前掩码。所以同一时刻有多个通道变化时，它们**合并成同一条带掩码的边沿**：你不会为“通道 3 上升 + 通道 7 下降”写两行，而是写一行新掩码，把这两个变化一起带上。这是该格式最省存储、也最省逻辑的地方——FSM 永远不需要做“读-改-写”位操作，只需把整行掩码拍进输出寄存器。

9.2.1 三块边沿表存储的精确布局

边沿表由三块按相同地址 (`edge_index`, 0–1023) 索引的分布式 RAM 组成。它们都标了 (`* ram_style="distributed" *`):

Verilog

```
(* ram_style = "distributed" *) reg [TICK_WIDTH-1:0] tick_mem
→ [0:MAX_EDGES-1];           // 32 位绝对基准 tick
(* ram_style = "distributed" *) reg [COEFF_BITS-1:0] coeff_mem
→ [0:MAX_EDGES-1];           // 64 位打包系数
(* ram_style = "distributed" *) reg [CHANNEL_COUNT-1:0] mask_mem
→ [0:MAX_EDGES-1];           // 62 位输出掩码
```

tick_mem[edge] 32 位无符号绝对 tick。这是这一行边沿在**未扫描**时应当触发的 tick。它是仿射公式里的 **base**。

coeff_mem[edge] 64 位，横向切成 4 个有符号 16 位系数：slot *j* 的系数在 `coeff_mem[edge][j*16 +: 16]`。即 `[15:0]`=slot0、`[31:16]`=slot1、`[47:32]`=slot2、`[63:48]`=slot3。未被任何 slot 影响的边沿，这 64 位全 0。

mask_mem[edge] 62 位输出掩码，bit *k* = 该行触发后 `channels[k]` 的电平。

9.2.2 有效 tick 的计算：仿射修正

运行时，这一行真正触发的时刻不是裸的 `base`，而是把 `slot` 当前值按系数仿射叠加后的**有效 tick**：

```
effective_tick = base + ( coeff_0*slot_0 + coeff_1*slot_1 +
    coeff_2*slot_2 + coeff_3*slot_3 ) » COEFF_FRAC_BITS
```

其中 `base = tick_mem[edge]`, `coeff_j = coeff_mem[edge][j*16 +: 16]` (有符号), `slot_j` 来自当前扫描点。 `COEFF_FRAC_BITS=8` 意味着系数是 Q8 定点：系数 256 表示“slot 每 +1, 这条边沿右移 1 个 tick”；系数 128 表示“slot 每 +1 右移半个 tick” (移位时截断)。负系数表示边沿随 slot 增大而**左移** (提前)。

9.2.3 逐位算一遍：一个真实的边沿行

设我们要写一条边沿：在基准 tick 1000 处，让通道 0 和通道 5 置高、其余清零；并让这条边沿被 slot 2 以“每 +1 个 slot 单位右移 1 个 tick”的方式扫描，其余 slot 不影响它。

1. **tick_mem**: `base = 1000`，即 32 位 `0x000003E8`。
2. **mask_mem**: 通道 0、5 为 1，其余 0。 `(1<0)|(1<5) = 0x21`，即 62 位里低 6 位为 `100001`，写成 `mask_mem[edge] = 62'h0000_0000_0000_0021`。
3. **coeff_mem**: 只有 slot2 有系数，“右移 1 个 tick / slot 单位” $\Rightarrow$ Q8 系数 = $1 \times 256 = 256 = 0x0100$ 。打包成 64 位：

- `slot0 = [15:0] = 0x0000`
- `slot1 = [31:16] = 0x0000`
- `slot2 = [47:32] = 0x0100`
- `slot3 = [63:48] = 0x0000`

即 `coeff_mem[edge] = 64'h0000_0100_0000_0000`。

运行时若当前扫描点给出 `slot2 = 7` (其余 slot=0)，则 `effective_tick = 1000 + (256*7) » 8 = 1000 + 1792 » 8 = 1000 + 7 = 1007`。也就是这条边沿在该扫描点上被推迟了 7 个 tick ($7 \times 20\text{ns} = 140\text{ns}$)。换一个扫描点 `slot2 = 50`，则 `1000 + (256*50) » 8 = 1000 + 50 = 1050`。可见 Q8 系数 256 恰好实现“slot 值 = tick 偏移量”的整数映射，这正是上位机做 duration/delay 扫描时最常用的系数。

为什么系数是有符号的

delay 扫描经常需要让某条边沿**提前** (slot 增大 $\Rightarrow$ tick 减小)，所以 `COEFF_WIDTH` 必须含符号位，移位与加法在 RTL 里按有符号做。16 位有符号 Q8 的系数范围约 ± 128 tick/slot 单位 ($\pm 128 \times 256$ 的整数部分)，对一次拍摄的扫描跨度足够。

9.3 为什么用分布式 RAM (LUTRAM) 而不是 BRAM

三块边沿表存储以及扫描点表都强制 `ram_style="distributed"`，即综合成 LUTRAM (用 SLICEM 里的 LUT 当小 RAM)，而**不是** Block RAM。这是本核心最重要的工程取舍，原因如下。

运行期读侧只有一个地址 FSM 在一次拍摄中，任何时刻只需要读**当前那一行** (`tick_mem[edge_index] / coeff_mem[edge_index] / mask_mem[edge_index]`)。单读口意味着 LUTRAM 的“

组合异步读”恰好够用——给出地址，同拍就能拿到行内容，无需 BRAM 的”地址寄存一拍、下一拍出数据”的同步读时序。

多读口在 Artix-7 上很贵 如果把读侧设计成需要同时读多行（例如同时看当前行和下一行），BRAM 每端口只有一个地址，要多读口就得复制整块 BRAM 或加复杂的端口仲裁。35T 上 BRAM 资源本就有限，复制 $1024 \times (32+64+62)$ 位的表很快吃光。LUTRAM 用器件里大量富余的 SLICEM LUT，单读口场景下反而更省、更简单。

元数据另行影子化，避开了多读需求 见下一节——”第一条边沿、循环起点、终止 tick”这些一次拍摄里要反复参照的元数据，不是运行期再去 RAM 里随机读，而是在**上传期**就锁存进普通寄存器。这样 RAM 真的只剩”当前行”这一个读地址，单读口 LUTRAM 完全成立。

小器件 + 无仿真器的双重压力 因为仓库**没有 Verilog 仿真器**，设计被刻意压向”逻辑越简单越能用 Python 契约测试与行为仿真覆盖”。LUTRAM 单读口、组合读、整行拍进输出寄存器，这套读通路足够简单到能在 Python 端逐 tick 复刻验证（参见仓库的 host↔RTL 行为一致性测试）。

改 RAM 风格前先想清楚

如果你把 `MAX_EDGES` 大幅调大（例如 8192），LUTRAM 可能开始挤占太多 SLICEM。这时再考虑改成 BRAM，但**必须**同时把读通路重构成同步读（地址提前一拍给出），否则时序会错一拍。不要只改 `ram_style` 属性而不改读时序。

9.4 读通路 + 影子元数据

9.4.1 运行期只读当前一行

一次拍摄中，FSM 维护一个 `edge_index` (0–1023)。它只做三件事的循环：用 `edge_index` 组合读出 `tick_mem/coeff_mem/mask_mem` 当前行，算出 `effective_tick`，当 32 位 tick 计数器到达 `effective_tick` 时把 `mask_mem[edge_index]` 拍进输出寄存器并 `edge_index++`。输出即寄存器：

Verilog

```
assign out = state_mask;    // 输出寄存器本身就是”当前掩码”
```

9.4.2 为什么要影子寄存器

但一次拍摄还需要几样**全局元数据**，它们不属于”当前行”，却要在运行中反复参照：

第一条边沿 (first-edge) 拍摄启动瞬间应当先把输出设成哪张掩码（起始电平）。

循环起点 (loop-start) 若 `repeat_forever` 或循环，回绕时要跳回的边沿索引/tick。

终止 tick (final-tick) 整个序列结束的 tick，用来判定一拍何时收尾。

如果运行期临时去 RAM 里随机读这些行（地址可能是 0、可能是循环起点、可能是末行），就需要**额外的读地址**，破坏”单读口”前提。解决办法是：在**上传期**（reset 拉高、随 `prog_we` 边沿写入时）就把这些元数据锁存进普通寄存器 `*_shadow`。

影子化的妙处

影子寄存器在上传期写一次、运行期只读。于是运行期对 RAM 的读地址永远只有 `edge_index` 一个。这就是即便整张表有 1024 行、也**完全不需要多读口 RAM** 的根本原因——元数据被搬到了寄存器，RAM 只剩”流式顺序读当前行”这一种访问模式。

改 RTL 时务必记住：任何“运行期需要随机访问某一行”的新需求，都应优先考虑能不能在上传期算好、锁进影子寄存器，而不是给 RAM 加读口。

9.5 时钟与对齐

时钟 真实硬件 50 MHz，故 **1 tick = 20 ns**。RTL 里所有时间都以 tick 计数。

量化 上位机在上传之前就把所有 duration/delay/扫描点按 $1e9/\text{clock_hz}$ (50 MHz 下 = 20 ns) 量化成整数 tick。RTL 端不做任何浮点或单位换算，只比较整数 tick。

量化误差 每条边沿的量化误差 $\leq$ **半个 20 ns tick** (10 ns)，因为量化是四舍五入到最近 tick。

计数器覆盖范围 32 位 tick 计数器在 50 MHz 下覆盖约 $2^{32} * 20\text{ns} \approx 85.9\text{ s}$ 。单次序列只要总时长 < 85.9 秒就不会溢出。

启动延迟 经 VIO 同步的 start 通路会引入一个**小的常数启动延迟**，它是**固定偏移**，**不是**比例因子——即它平移所有边沿的绝对触发时刻，但不拉伸边沿之间的相对间隔。做相对时序校准时只需关心边沿间隔，不必担心这个常数。

改时钟频率怎么办

若把硬件时钟改成非 50 MHz，**只需**让上位机用新的 **clock_hz** 重新量化；RTL 一行都不用改，因为 RTL 只认整数 tick。但要重新核对：新频率下 32 位计数器覆盖的总时长 ($2^{32}/f$) 是否仍 $\geq$ 你最长的序列。

9.6 同步器与边沿检测：让 VIO 拨一下只写一次

所有从 VIO (异步、由 Tcl 在 Vivado 里随手 toggle) 进来的控制信号——reset、start、prog_we、scan_prog_we、bus_prog_we——都各自配一组**三级寄存器**：*_meta、*_sync、*_prev。

Verilog

```
// 每个 VIO 输入的标准三级打拍 + 边沿检测
xxx_meta ≤ xxx;           // 第一级：吸收亚稳态
xxx_sync ≤ xxx_meta;      // 第二级：进入时钟域的干净电平
xxx_prev ≤ xxx_sync;      // 保存上一拍的同步值
wire xxx_event = xxx_sync ≠ xxx_prev; // 同步值发生跳变 = 一次事件
```

meta + sync 两级 标准双触发器同步器，把 VIO 来的异步电平安全搬进 50 MHz 时钟域，吸收亚稳态。

prev + event ***_event = (*_sync ≠ *_prev)**：只在同步后的电平**发生跳变**的那一拍为高。

为什么要 event 而不是电平 VIO 的 toggle 是一次“把某位翻一下”。如果直接用电平当写使能，VIO 拨一下后电平会保持，导致连续多拍重复写。用 *_event (跳变检测) 保证 **一次 toggle 精确产生一次写**——例如 prog_we_event 为高的那一拍才把 prog_addr/prog_tick/prog_tick_coeffs/prog_mask 写进对应 RAM 一行。

加新 VIO 控制信号的固定套路

任何新增的、从 VIO 进来的“动作型”信号都**必须**走这套 meta/sync/prev + event 模板，并用 event 做一次性触发。直接拿 VIO 电平当写使能 / 启动信号是经典 bug——会随 VIO 刷新率重复触发。

9.7 上传契约：六步把一张表灌进 FPGA

上位机 (Zou\lab_control\nneutral_atom\devices\fpga_pulse_streamer.py 经 VIO Tcl) 按**固定六步**把一张边沿表 + 扫描配置写进核心。顺序不能乱，因为写 RAM 必须在 reset 拉高期间、且元数据影子锁存依赖 prog_we 边沿。

1. **拉高并保持 reset**。整个上传过程 reset 始终为高，FSM 不跑，RAM 写口独占。
2. **设置 prog_count** (本次表的边沿条数, ≤ 1024)。
3. **逐条边沿写入**：对每一行设置 prog_addr (目标 edge_index)、prog_tick (写 tick_mem)、prog_tick_coeffs (64 位打包, 写 coeff_mem)、prog_mask (62 位, 写 mask_mem), 然后 toggle prog_we。prog_we_event 那一拍完成该行三块 RAM 的写入, 同时按需更新 first-edge / loop-start / final-tick 的影子寄存器。
4. **可选：写扫描点表与总线段**。scan_prog_* (toggle scan_prog_we 写 scan_value_mem, 每点 128 位打包 4 个 slot) 与 bus_prog_* (toggle bus_prog_we 写 DAC 总线段)。无扫描时可跳过。
5. **设置循环 / 扫描配置**：repeat_forever、loop_* (循环起点/终点等)、scan_enable、scan_count (扫描点数)。
6. **释放 reset, 然后脉冲 start**。release 后拨一下 start, start_event 启动拍摄；FSM 从 first-edge 影子电平开始, 按 edge_index 流式读表跑完整个序列 (或按扫描/循环配置反复跑)。

9.7.1 扫描点表的比特布局

scan_value_mem[0:MAX_SCAN_POINTS-1] 每行 SLOT_BITS = 128 位, 是一个扫描点的 4 个 slot 值打包: slot j 占 scan_value_mem[point][$j*32 + :$ 32] (完整 32 位 tick 宽)。运行时, 当前扫描点的 slot_j 就从这里取, 喂给前述 effective_tick 公式 (以及 DAC 的 value_select 通路, 取其低 BUS_WIDTH 位作 DAC 码)。

为什么每 slot 给满 32 位 slot 值可能被当作 tick 偏移量 (duration/delay 扫描, 需 32 位 tick 量程), 也可能被当作 DAC 码 (取低 10 位)。给满 32 位让同一张扫描点表能同时服务两种用途, 无需为不同 slot 用途切换布局。

为什么扫描点也用分布式 RAM 运行期每个扫描点只读一次、且只读当前点, 仍是单读口顺序访问, LUTRAM 同样最划算。

9.7.2 一个完整上传的数值小例

设一张 3 行边沿表, 无扫描 (scan_enable=0):

1. reset 拉高。
2. prog_count = 3。
3. 写行 0: prog_addr=0, prog_tick=0, prog_tick_coeffs=64'h0, prog_mask=62'h0 (全灭, 起始电平) → toggle prog_we。写行 1: prog_addr=1, prog_tick=50 ($=50 \times 20\text{ns} = 1\mu\text{s}$), coeffs=0, prog_mask=62'h1 (通道 0 置高) → toggle。写行 2: prog_addr=2, prog_tick=100 ($=2\mu\text{s}$), coeffs=0, prog_mask=62'h0 (通道 0 拉低) → toggle。
4. 无扫描点表 / 总线段, 跳过。

5. `repeat_forever=0, scan_enable=0`, final-tick 影子来自最后一行。

6. 释放 reset, 脉冲 `start`。

运行结果：通道 0 在 $t = 1\mu s$ 升起、 $t = 2\mu s$ 落下，得到一个 $1\mu s$ 宽脉冲。若要扫描这个脉冲宽度，只需把行 2 的 `coeff_mem[2][slot0]` 设成 256 (Q8 的 1 tick/slot 单位)、`scan_enable=1`、并往 `scan_value_mem` 写若干点的 slot0 值（每点一个 tick 偏移），行 2 的有效落沿就会随扫描点平移，脉宽随之扫描——上沿不动（行 1 系数为 0），下沿动，正是 duration 扫描。

改上传契约时的不变量

五条铁律：(1) 写任何 RAM 必须在 reset 高电平期间；(2) 每行用一次 `prog_we` toggle（靠 event 一次性写）；(3) 影子元数据只在上传期更新，运行期只读；(4) 系数按 Q8 有符号、每 slot 16 位、共 64 位打包；(5) 扫描点每 slot 满 32 位、共 128 位打包。任何新字段都应沿用“上传期 toggle 写 + 运行期单读口”的范式，并同步更新 Python 端打包器（`_pack_signed` 等）与 VIO 宽度契约测试，否则 RTL 与上位机会静默漂移。

10

N-slot 仿射扫描引擎（深入）

本章深入剖析 `fpga/pulse/_streamer/zlc/_pulse/_streamer.v` 中的扫描引擎。这套引擎彻底取代了早期的双变量 (x, y) 仿射引擎——现在硬件里不再存在 x/y 任何痕迹，取而代之的是 `NUM_SLOTS` 个命名 slot。读完本章，一个硬件工程师应当能够独立读懂、修改并扩展这套 RTL：每个信号的位宽为什么是那样、为什么用分布式 RAM (distributed RAM) 单读口、每个 task/function 的精确位布局，以及在 Artix-7 35T (xc7a35tfgg484-2, 约 20.8k LUT) 的资源预算下这些取舍的动机。

10.1 为什么是“仿射 + 流式扫描点表”而不是 run-length 重写

spec 最初要求的是 per-channel run-length (逐通道游程编码) + dual-bank (双缓冲) 的全新存储方案。我们放弃了它，改为把旧的双变量仿射引擎泛化成 N 个 slot 的流式扫描点表。原因全部围绕 35T 的现实约束：

- **存储目标已达成。** 仿射模板 + 流式扫描点表的存储复杂度是 $O(\text{edges}) + O(\text{points} \times \text{slots})$ ：边沿模板只存一份，扫描点表每行只存 `NUM_SLOTS` 个 32 位整数。这恰好是 spec 的核心存储目标，但不需要 run-length 那套额外的状态机和地址逻辑。
- **LUT 最省。** run-length/dual-bank 需要双份边沿缓冲区、bank 切换逻辑、以及每通道独立的游程计数器；在 35T 上这会迅速吃掉 LUT 预算。仿射方案复用一条 `zlc_effective_tick` 计算链，只在边沿表每行追加几个系数列。
- **复用分布式 RAM。** 所有大表 (`tick_mem`、`scan_value_mem`、bus 段表) 都标注 `(* ram_style = "distributed" *)`，映射到 LUT 内的分布式 RAM。分布式 RAM 是单读口的——这是一条贯穿全设计的硬约束：每个表在每个时钟周期只能读出一个地址。FSM 的所有遍历逻辑都必须围绕“每周期一次读”来组织。
- **可验证。** 在仓库里没有 Verilog 仿真器 (无 iverilog/cocotb) 的前提下，仿射方案能被纯 Python 的契约测试 (位宽对齐、探针打包) + 行为仿真 (逐 tick 复刻 RTL 数学) 覆盖。run-length 的复杂状态机就很难用 Python 行为模型忠实复现。

曾尝试的单通道 run-length 重写已被放弃 (单 BRAM 读口下每个边界要重装 62 个通道约 $23.6 \mu\text{s}$ ，无法无缝，见末章“JTAG-to-AXI 边沿表加载器”)。edge-table 引擎是当前唯一引擎，并通过片上加载器接到 JTAG-to-AXI 控制路径。

10.2 仿射公式与 zlc_effective_tick 内部实现

引擎的全部数学浓缩在一个公式里。对于当前扫描点的 slot 向量，FPGA 把每个边沿的“基准 tick” 平移成有效 tick：

text

```
effective_tick = base_tick + (sum_j coeff_j * slot_j) >>> COEFF_FRAC_BITS
```

其中 $\gg$ 是**算术右移**（保留符号）， $\text{COEFF_FRAC_BITS} = 8$ 固定，即系数是 8 位小数的定点数：系数 256 表示“每单位 slot 平移 1 个 tick”（ $256 = 1 \ll 8$ ）。

10.2.1 参数与位宽

模块顶部的参数（fpga/pulse_streamer/zlc_pulse_streamer.v 第 30–34 行附近）：

$\text{TICK_WIDTH} = 32$ tick 计数器与所有 tick 量的位宽。50 MHz / 20 ns 一个 tick，32 位足够覆盖极长序列。

$\text{NUM_SLOTS} = 4$ 同时可扫的命名 slot 数。顶层例化用 4，足以覆盖同时扫 3–4 个接口（duration / delay / DAC 值）。

$\text{COEFF_WIDTH} = 16$ 每个仿射系数的位宽，**有符号**。16 位定点（8 位整数 + 8 位小数的范围内）足够表达常见的 tick/slot 斜率。

$\text{COEFF_FRAC_BITS} = 8$ 定点小数位，固定为 8。

由此派生的本地参数（第 89–91 行附近）：

$\text{COEFF_BITS} = \text{NUM_SLOTS} * \text{COEFF_WIDTH}$ 一行打包系数的总位宽。 $\text{NUM_SLOTS}=4$ 时为 64 位。

$\text{SLOT_BITS} = \text{NUM_SLOTS} * \text{TICK_WIDTH}$ 一个 slot 向量的总位宽。 $\text{NUM_SLOTS}=4$ 时为 128 位。

$\text{ACC_WIDTH} = \text{TICK_WIDTH} + \text{COEFF_WIDTH} + 4 = 52$ 累加器位宽。**为什么是 +4?** 累加器要容纳 NUM_SLOTS 个 $\text{coeff_i} (16b) * \text{slot_value_i} (32b)$ 的乘积之和。单个乘积最宽 48 位，求和最多 NUM_SLOTS 项；+4 位的余量（ $\log_2(16) = 4$ ）保证即便 16 个 slot 也不溢出，给未来增大 NUM_SLOTS 留了头寸，同时不浪费过多 LUT。

10.2.2 打包约定 (little-endian)

slot 系数和 slot 值都**小端打包**进一个宽向量，host 与 RTL 必须严格一致：

- 系数： $\text{coeffs}[j*16 +: 16]$ 取出第 j 个 slot 的 16 位有符号系数。
- slot 值： $\text{slots}[j*32 +: 32]$ 取出第 j 个 slot 的 32 位有符号值。

这意味着 slot 0 在最低位。host 端(Zou_lab_control/neutral_atom/devices/sequencer.py) 的打包函数与 VIO 探针位宽契约测试都锁定这一布局——任何一边改了端序，位宽对齐测试 `test\_...\_vio\_widths\_match\_python\_generator` 会立刻失败。

10.2.3 function 主体

`zlc_effective_tick` 是一个纯组合 function (第 189–207 行附近):

Verilog

```
function [TICK_WIDTH-1:0] zlc_effective_tick;
    input [TICK_WIDTH-1:0]    base_tick;
    input [COEFF_BITS-1:0]    coeffs;
    input [SLOT_BITS-1:0]     slots;
    reg signed [ACC_WIDTH-1:0] acc;
    reg signed [COEFF_WIDTH-1:0] coeff_i;
    reg signed [TICK_WIDTH-1:0] slot_value_i;
    reg signed [ACC_WIDTH-1:0] total;
    begin
        acc = {ACC_WIDTH{1'b0}};
        for (slot_i = 0; slot_i < NUM_SLOTS; slot_i = slot_i + 1) begin
            coeff_i      = coeffs[slot_i*COEFF_WIDTH +: COEFF_WIDTH];
            slot_value_i = slots[slot_i*TICK_WIDTH +: TICK_WIDTH];
            acc = acc + coeff_i * slot_value_i;    // signed * signed
        end
        total = $signed({1'b0, base_tick}) + (acc >>> COEFF_FRAC_BITS);
        zlc_effective_tick = total[TICK_WIDTH-1:0];
    end
endfunction
```

要点:

- `coeff_i` 与 `slot_value_i` 都声明为 `signed`, 所以 `coeff_i * slot_value_i` 是**有符号乘法**——slot 值可以是负的 (向左平移边沿), 系数也可以是负的。
- `acc` 累加全部 `NUM_SLOTS` 项, 再整体 `>>> 8` 还原定点。**先累加后移位**很关键: 每项单独移位会丢掉小数位的累积精度。
- `base_tick` 用 `$signed({1'b0, base_tick})` 零扩展成有符号, 与移位后的 `acc` 相加得到 `total`; 最终截回 `TICK_WIDTH` 位。base 是无符号 tick, 加上可正可负的平移量。
- 这是一个**展开的 for 循环** (综合时按 `NUM_SLOTS` 完全展开成 `NUM_SLOTS` 个乘加), 不是时序逻辑——它在被调用处作为纯组合表达式被实例化。

该 function 在多处被调用: 组合的 `current_edge_tick` (第 173 行) 用当前 `slot_active` 实时算出当前边沿的有效 tick 供时间比较; bus 段的 `eff_tk_start/eff_tk_stop` (第 267–268 行); 以及 FSM 在切换扫描点时重算 `final_tick / loop_end_active / 首边沿`。

10.3 流式扫描点表与无缝迭代 FSM

10.3.1 两份上传: 模板 + 扫描点表

host 只上传**一份边沿模板** + **一份流式扫描点表**:

边沿模板 `tick_mem` (基准 tick) + 打包的每边沿系数 (`prog_tick_coeffs`) + 62 位通道掩码。每个边沿存 `base_tick` 加 `NUM_SLOTS` 个有符号系数。

扫描点表 `scan_value_mem [0:MAX_SCAN_POINTS-1]`, 每行是 `SLOT_BITS` 宽 (128 位 = 4 个 32 位 slot 值), 标注分布式 RAM、**单读口** (第 100 行)。第 `p` 行就是第 `p` 个扫描点的 slot 向量。

为什么这样分：模板描述“形状”，扫描点表描述“每个点把哪些 slot 设成什么值”。两者正交，存储量分别是 $O(\text{edges})$ 和 $O(\text{points} \times \text{slots})$ ，互不相乘。

10.3.2 无缝遍历的关键寄存器

scan_point_index 当前扫描点下标，`SCAN_ADDR_WIDTH+1` 位（第 129 行）。多出的 1 位用于“是否已到末点”的越界比较，避免回绕误判。

active_scan_count 本次运行的有效扫描点数（第 128 行），运行开始时从 `scan_count` 锁存。

slot_active 当前点的 slot 向量寄存器，`SLOT_BITS` 宽（第 127 行）。`zlc_effective_tick` 全靠它驱动。

final_tick / loop_end_active 当前点解析后的“模板末 tick”与“循环末 tick”，都是 `slot_active` 代入仿射公式的结果（第 112、126 行）。

_shadow 寄存器 `final_tick_shadow / first_tick_shadow / loop_start_tick_shadow` 以及对应的 `*_coeffs_shadow`，保存模板的**基准值与系数**（与具体扫描点无关），在切点时用来对新 slot 向量重新代入公式。

10.3.3 无缝切点的 FSM 逻辑

核心在 `time_count ≥ final_tick` 的分支（第 476–483 行附近）。当前点的整个模板跑完后（时间计数到达有效末 tick），若还有下一个点：

Verilog

```
end else if (time_count ≥ final_tick) begin
  if (scan_enable_active && (scan_point_index + 1'b1) <
    ↪ active_scan_count) begin
    zlc_bus_start_table(scan_value_mem[next_scan_addr]);
    scan_point_index ≤ scan_point_index + 1'b1;
    slot_active      ≤ scan_value_mem[next_scan_addr];
    final_tick       ≤ zlc_effective_tick(final_tick_shadow,
                                          final_coeffs_shadow,
                                          scan_value_mem[next_scan_a]
                                          ↪ ddr]);
    loop_end_active  <= zlc_effective_tick(loop_end_tick,
    ↪ loop_end_coeffs,
                                          scan_value_mem[next_scan_a]
                                          ↪ ddr]);
    if (zlc_effective_tick(first_tick_shadow, first_coeffs_shadow,
                          scan_value_mem[next_scan_addr]) = 0) begin
      // 首边沿落在 tick 0, 立即施加该点的初始掩码
    end
    ...
  end
end
```

逐条说明：

- `next_scan_point_index = scan_point_index + 1`，其低位 `next_scan_addr` 用作 `scan_value_mem` 的读地址（第 169–170 行），注意整个切点过程**对 `scan_value_mem[next_scan_addr]`**

的读出值反复使用——因为分布式 RAM 单读口，同一周期对同一地址的多次引用综合后共享一个读端口，这是合法且省资源的。

- `slot_active` 被更新为新点的 slot 向量；但**新点的** `final_tick` / `loop_end_active` / 首边沿 必须用**即将载入的新向量**（即直接用 `scan_value_mem[next_scan_addr]`，而不是还没锁存的 `slot_active` 寄存器）来计算。这避免了非阻塞赋值（NBA）时序坑：如果用 `slot_active`，它在本周末才更新，本周期内重算会用到旧值。
- `zlc_bus_start_table(...)` 也显式传入新的 slot 向量，使 bus (DAC) 引擎在进入新点的**第一个周期**就用正确的 slot 解析段，无需等 `slot_active` 锁存。
- 若首边沿的有效 tick 解析为 0，说明该点在 tick 0 就有边沿，立即施加其初始掩码；否则等时间计数走到。
- **每个扫描点触发一次相机** (camera trigger)，所以扫 N 个点采集 N 帧。

跑完最后一个点（第 495–500 行附近）则回绕到点 0（或在非扫描模式下回到全零 slot），同样用新向量重算各 tick。整个过程**不停流水线**：硬件在点与点之间无须等 host 重新上传，故称“无缝”。

10.4 一个完整的数值例子：扫 duration s0 三个点

设想一个简单序列：通道 `cooling` 在某 period 内点亮，该 period 的 duration 被绑定到 slot 0 (s0)。被扫的 duration 让这个 period 的**仿射系数非零**——具体是 256/tick (256 = 1«8，即“每多 1 个 tick 的 duration，后续边沿平移 1 个 tick”)。一个 delay 若也被绑定，会叠加它自己的系数（仿射可加性）。

设模板有两个相关边沿：

- 边沿 A: period 起点, `base_tick = 100`, 系数 `coeff = [0, 0, 0, 0]` (不随 s0 移动)。
- 边沿 B: period 终点 (cooling 关断), `base_tick = 100`, 系数 `coeff = [256, 0, 0, 0]` (随 s0 线性移动)。

扫描点表三行（只用 s0，其余 slot = 0）：

点 0 `slots = [50, 0, 0, 0]`

点 1 `slots = [80, 0, 0, 0]`

点 2 `slots = [120, 0, 0, 0]`

边沿 B 的有效 tick 按公式 `effective_tick = base + (coeff0*s0) >> 8`：

- 点 0: $100 + (256 \times 50 \gg 8) = 100 + (12800 \gg 8) = 100 + 50 = 150$ 。
- 点 1: $100 + (256 \times 80 \gg 8) = 100 + (20480 \gg 8) = 100 + 80 = 180$ 。
- 点 2: $100 + (256 \times 120 \gg 8) = 100 + (30720 \gg 8) = 100 + 120 = 220$ 。

于是 cooling 的关断边沿随扫描点从 tick 150 → 180 → 220 移动，而起点边沿 A 固定在 100: cooling 脉宽 (duration) 依次为 50、80、120 个 tick。**系数 256 的含义在此显形**：slot 值直接以 tick 为单位，乘 256 再右移 8 等于恒等映射，所以 s0 的数值就是 duration 的 tick 增量。若 s0 还要带亚 tick 缩放，可把系数设成非 256 的值（如 128 = 半 tick/单位）。

复合叠加

若该 period 内还有一个 delay 绑定到 s1，则 period 起点之后的边沿系数会变成 `[256, 256, 0, 0]` 之类——duration 的 256（来自 s0）与 delay 的 256（来自 s1）**相加**。这就是“delay 叠加它自己的系数，与 duration 仿射可加地复合”的精确含义：所有受影响边沿的有效 tick 都是各 slot 贡献

之和。

10.5 host 端安全边界（编译期拒绝）

并非任意时序都能塞进“一份模板 + 仿射系数”的结构。`compile_pulse_table_scan_runtime_program` (Zou_lab_control/neutral_atom/devices/sequencer.py) 在编译期把不可表达的情况直接拒绝，而不是生成错误的 HDL：

只接受仿射时序 slot 中的时序表达式必须是仿射的 (`base + sum coeff_j*s_j`)。非仿射的时序（例如 slot 的乘积、条件分支）无法编译成每边沿的常数，直接报错。

拒绝跨扫描点会换序的模板 边沿表只有一份固定的行序。如果某个扫描点会让两个边沿的先后顺序互换（即一行固定边沿序无法同时满足两个点矛盾的时序），编译器拒绝——一份固定边沿序不可能描述自相矛盾的时序。遇到这种情况，需要把序列切成多块 (chunks) 分别上传。

至少一个绑定 slot 硬件扫描要求至少绑定一个 duration/delay/DAC slot，否则“没有可扫的量”，报错。

扫描表行宽校验 每行的值数必须等于 `len(state.scan_slots)`，否则报告“第 N 行有 X 个值但有 Y 个 slot”。

这些拒绝是设计的一部分而非缺陷：它们把“硬件无法无缝执行的时序”在 host 侧拦下，保证上传到 FPGA 的程序一定能被仿射引擎正确流式执行。

10.6 repeat 是元数据，不是展开的行

重复不通过复制边沿行实现——那会浪费分布式 RAM。它是模板上的元数据：

整表无限重复 `repeat_forever = 1`。FSM 跑到 `final_tick` 后回绕到点 0 / tick 0，永远循环。

有限内层括号 用 `loop_start_addr` (循环起始边沿地址)、`loop_end_tick` (循环末 tick 的基准值)、`loop_count` (次数)，外加 `loop_end_coeffs` (循环末 tick 的仿射系数)。循环末同样随 slot 仿射移动：第 457 行 `loop_end_active ≤ zlc_effective_tick(loop_end_tick, loop_end_coeffs, scan_first_values)`，切点时也重算 (第 482 行)。

循环执行逻辑 (第 470–475 行附近)：当 `loop_count_active > 1 && loops_remaining > 1 && time_count ≥ loop_end_active`，把 `time_count` 跳回 `zlc_effective_tick(loop_start_tick_shadow, loop_start_coeffs_shadow, slot_active) + 1` (循环起点的有效 tick 加一)，并重启 bus 表。`running / done` 不由展开行数驱动，而由 `prog_count` 与当前 `scan_point_index` 决定：序列是否还在跑、跑到第几点，都是计数器比较，不是物理行的耗尽。

为什么省资源

把 repeat 做成元数据意味着：一个重复 1000 次的内层括号在 `tick_mem` 里仍然只占该括号那几行边沿，而不是 1000 份拷贝。在 35T 的分布式 RAM 预算下，这是“能不能放下一个真实实验序列”的分水岭。仿射系数让循环边界也能随扫描点平移，于是“扫一个 duration 的同时内层循环 N 次”这类组合无需任何额外存储即可表达。

11

模拟总线 DAC 引擎、value_select 与仿射总线 tick (深入)

本章深入剖析 `fpga/pulse/_streamer/zlc/_pulse/_streamer.v` 中独立于 62 位数字输出的**模拟总线 (bus) 引擎**。它驱动 4 路 10 位 DAC 总线, 支持边沿跳变、DDA 斜坡、扫描 DAC 值 (`value_select`), 以及与数字时序同步平移的仿射总线 `tick`。读完本章, 硬件工程师应当能够独立理解、修改并扩展这一引擎, 包括它为何这样组织内存、为何在 Artix-7 35T (`xc7a35tfgg484-2`, 约 20.8k LUT) 上仍然装得下、以及主机侧数据如何写入。

11.1 为什么需要一张独立的总线表

数字输出 (62 位 TTL) 使用**边沿表** (`tick_mem / mask_mem`): 每一行是一个绝对时刻 (`tick`) 与一个 62 位掩码。这种表达对 TTL 极其紧凑, 因为 TTL 在两个边沿之间保持不变, 整段时间只占边沿表里的一行。

但 DAC 不一样。一条缓慢的模拟斜坡如果展开成边沿表, 会逐个 `tick` 地写一行——一条跨越数千个 `tick` 的斜坡会瞬间塞满 1024 行的边沿表。因此 4 路 DAC (`da\_dipole`、`da\_bias\_y`、`da\_bias\_x`、`da\_bias\_z`, 由 `bus_out` 总线输出) 使用一张**独立的总线段 (bus-segment) 表**, 其中一段斜坡无论跨多少个 `tick` 都只占**一行**。普通 TTL 继续走 `tick/mask` 边沿表, 二者互不干扰, 各取所需。

一句话总结

TTL: 边沿表 ($O(\text{边沿数})$)。DAC: 段表 ($O(\text{段数})$), 斜坡 = 一行 + 硬件 DDA 内插)。两套表并行执行, 共享同一个 `time_count` 时钟域。

11.2 段内存的逐字段布局

总线段内存以一维数组形式存放, 索引范围 `[0:MAX_BUS_SEGMENT_ROWS-1]`。地址是**扁平化**的: 第 `i` 路总线、第 `k` 段的行地址为 `addr = i*MAX_BUS_SEGMENTS + k`。其中:

- `MAX_BUS_SEGMENTS = (1 << BUS_SEG_ADDR_WIDTH)`, 每路总线最多 64 段。
- `MAX_BUS_SEGMENT_ROWS = BUS_COUNT * MAX_BUS_SEGMENTS = 4 * 64 = 256`

行总容量。

每行由**多个平行的分布式 RAM**组成，每个字段一块 RAM。它们都带 (* ram_style = "distributed" *) 属性，原因见下文。逐字段含义：

bus_start_tick_mem / bus_stop_tick_mem 各 **TICK_WIDTH=32** 位。段的起始 / 结束基准时刻 (base tick)。edge 模式下两者相等；ramp 模式下 stop>start。

bus_start_value_mem / bus_stop_value_mem 各 **BUS_WIDTH=10** 位。段的起始 / 目标 DAC 码 (字面值)。当 value_select 启用时这两个字段被运行时扫描值**覆盖**。

bus_mode_mem 2 位。**BUS_MODE_EDGE=2'd1** (跳变到值)、**BUS_MODE_RAMP=2'd2** (DDA 内插)。第三种“hold”**不占段** (见下文)。

bus_value_select_mem **BUS_SEL_WIDTH=3** 位。DAC 值扫描选择器：0 = 用字面值；j+1 = 取 slot j 的低 10 位作 DAC 码。3 位足以编码“字面 + 最多 4 个 slot 引用”。

bus_start_tick_coeff_mem / bus_stop_tick_coeff_mem 各 **COEFF_BITS = NUM_SLOTS*COEFF_WIDTH = 4*16 = 64** 位。仿射 tick 系数，每个 slot 一个有符号 16 位系数，按 slot 序小端打包。让段的起止 tick 随被扫描的 duration/delay 平移。

每路总线的**有效段数**打包在端口 **bus_counts** 里，宽度 **BUS_COUNT*(BUS_SEG_ADDR_WIDTH+1) = 4*(6+1) = 28** 位——每路一个 7 位计数(6 位地址 + 1 位以表达“满 64”)。函数 **zlc_bus_count_at(i)** 把它拆出来：

Verilog

```
function [BUS_SEG_ADDR_WIDTH:0] zlc_bus_count_at;
    // 路 0: bus_counts[6:0]; 路 1: [13:7]; 路 2: [20:14]; 路 3: [27:21]
    0: zlc_bus_count_at = bus_counts[BUS_SEG_ADDR_WIDTH:0];
    1: zlc_bus_count_at =
        ↪ bus_counts[(2*(BUS_SEG_ADDR_WIDTH+1))-1:(BUS_SEG_ADDR_WIDTH+1)];
    ...
endfunction
```

为什么有 count 字段：256 行 RAM 在不同实验间会留下陈旧数据。运行时只执行 0..count-1 段，超出 count 的行 (旧程序残留) 永不读取。这样既不必每次清空 RAM，又保证不会误触发旧段。

为什么所有 RAM 都是 distributed 且单读口

注释明确写道：运行时读侧坚持**一个表地址**。Vivado 若看到多读口会综合出昂贵的多端口存储，在 Artix-7 上吃掉大量 LUT。256 行 × (32+32+10+10+2+3+64+64 位) 用 LUT 作分布式 RAM (每片 LUT 当一小块 RAM) 实现，比强行用 BRAM 多读口省得多，也避开 BRAM 的读延迟。每个时钟每路总线只读**一个段地址 bus_runtime_addr**，这是设计能塞进 35T 的关键约束。修改时务必保持“运行时一拍只读一个段地址”这条不变量。

11.3 三种模式与 DDA 斜坡

应用一个段由任务 **zlc_bus_apply_segment(i, addr, slot_vec)** 完成。它先解析 DAC 值 (见 value_select 节) 和有效 tick (见仿射 tick 节)，然后按模式分派：

edge (边沿/跳变) **start_tick = stop_tick**。到达该 tick 时直接把 **bus_value_active[i]** 置为目标值，**bus_ramp_active[i]** 清零。代码走 **else** 分支：**bus_value_active[i] ≤ eff_val_stop**。一次性跳变，不占内插逻辑。

ramp (斜坡) `stop_tick > start_tick`。启动一个 DDA (数字差分分析器) 累加器, 在 start 到 stop 之间每步把 `bus_value_active` 增/减 1, 到 stop_tick 时**吸附**到精确目标值。无论跨多少 tick, 斜坡只占**一段**。

hold (保持) **不产生新段**。当一路总线在某时刻没有新段可应用时, 它保持当前 `bus_value_active` 或继续正在进行的斜坡。hold 不是存在 RAM 里的模式值, 而是“没有段在这一刻触发”这一状态的自然结果。

DDA 斜坡的实现 (在 `zlc_bus_step` 里每拍推进) 使用三个状态:

`bus_ramp_delta[i]` 数值跨度 = $|\text{stop_value} - \text{start_value}|$, 即斜坡要走的 DAC 码总数。同时记录方向 `bus_ramp_dir_up[i]`。

`bus_ramp_denom[i]` 时间跨度 = `stop_tick - start_tick`, 即可用的步数 (denominator)。

`bus_ramp_accum[i]` 累加器, 宽 `TICK_WIDTH+BUS_WIDTH+1` 位, 初值 0。

每个在斜坡区间内的 tick 执行: `accum_next = accum + delta`; 若 `accum_next ≥ denom` 则 `accum ≤ accum_next - denom` 并把 `bus_value_active` 朝目标移 1 (带边界保护, up 时不超过 target, down 时不低于 target); 否则只 `accum ≤ accum_next`。这正是 Bresenham 式整数内插: 跨 `denom` 个 tick 恰好累计 `delta` 次增量, 无需乘除。到 `time_count ≥ bus_ramp_stop_tick` 时强制 `bus_value_active ≤ bus_ramp_target` 并结束斜坡, 消除任何舍入残差。

斜坡算例

start_value=100、stop_value=900 (delta=800) , start_tick=1000、stop_tick=1400 (denom=400)。每拍 `accum += 800`。第 1 拍 `accum=800 < 1400?` `denom=400`, $800 \geq 400 \rightarrow \text{accum}=400, \text{value}+1$; 第 2 拍 `accum=1200 \geq 400 \rightarrow \text{accum}=800, \text{value}+1`, $800 \geq 400$ 仍只增一次/拍 (每拍至多 +1)。平均每拍 $800/400 = 2$ 个码, 400 拍走完 800 码, 结束时吸附到 900。整段只占一行 RAM。

11.4 value_select: DAC 值扫描路径与 slot_vec/NBA 时序坑

`value_select` 是**扫描 DAC 码**的通路。读自 `bus_value_select_mem[addr]` (记为 `sel`):

Verilog

```
sel = bus_value_select_mem[addr];
if (sel != 0) begin
    // 扫描 DAC 码: 取 slot (sel-1) 的低 BUS_WIDTH 位
    eff_val_start = slot_vec[(sel - 1)*TICK_WIDTH +: BUS_WIDTH];
    eff_val_stop = eff_val_start;
end else begin
    eff_val_start = bus_start_value_mem[addr];
    eff_val_stop = bus_stop_value_mem[addr];
end
```

`sel == 0` 用字面值 (RAM 里的 start/stop value), 普通固定 DAC 电平。

`sel == j+1` 取**当前扫描点**的 slot 向量第 `j` 个 slot 的低 10 位作 DAC 码。每个 slot 在向量里占 `TICK_WIDTH=32` 位 (与 tick 同宽, 便于复用 SLOT_BITS 打包), 所以位选是 `slot_vec[(sel-1)*TICK_WIDTH +: BUS_WIDTH]`——以 32 位为步距定位 slot, 再取低 10 位。这样一路 DAC 电平就**无缝**跟随扫描点逐点变化, 不必为每个扫描点重写整张总线表。

关键时序坑——为什么 slot_vec 必须显式传参: 注意 zlc_bus_apply_segment 接收 slot_vec 作为**显式输入**，而不是在任务内部读全局寄存器 slot_active。原因在扫描点边界：当流水跨入一个新扫描点时，slot_active 是用**非阻塞赋值 (NBA)** 更新的，它在当前这一拍**尚未锁存**新点的值。如果任务此刻去读 slot_active，会读到**上一个点**的旧 slot 向量，于是新点第一拍的 DAC 码就错了。

解决办法是把“本应生效的 slot 向量”显式喂进来：

- 序列**启动**时 (zlc_bus_start_table) 传 scan_first_values (第一个扫描点的 slot 向量)。
- 跨**推进**到下一个扫描点时传 scan_value_mem[next] (下一个点的 slot 向量，已是组合可读的 RAM 输出)。
- 在一个扫描点**内部**逐段推进 (zlc_bus_step 内) 则传 slot_active——此时它已稳定锁存，读它是安全的。

这样新点的第一拍就拿到正确的 slot 向量，DAC 值“零延迟”切换。修改这部分时，凡是在**扫描点边界**调用 apply，都必须传“目标点”的 slot 向量而非 slot_active，否则会引入一拍的值错位。

11.5 仿射总线 tick: zlc_bus_seg_start 与同时扫 DAC 值 + duration + delay

DAC 值扫描解决了“扫电平”，但要让 DAC 段在**扫描 duration/delay**时仍与数字边沿**同步平移**，段的 tick 本身也必须是仿射的。这就是 bus_start_tick_coeff_mem / bus_stop_tick_coeff_mem (各 64 位 = 4 slot × 16 位有符号系数) 的用途。

有效 tick 由共享函数 zlc_effective_tick(base, coeffs, slot_vec) 计算，公式为：

$$\text{effective_tick} = \text{base_tick} + (\text{sum_j } \text{coeff_j} * \text{slot_j}) \gg \text{COEFF_FRAC_BITS}$$

其中 COEFF_FRAC_BITS=8(系数是 Q8 定点),内部累加器 ACC_WIDTH = TICK_WIDTH+COEFF_WIDTH+4 位有符号,最后右移 8 位、加上无符号 base、截到 32 位 tick。apply 任务对每段同时算 start 和 stop 的有效 tick:

Verilog

```
eff_tk_start = zlc_effective_tick(bus_start_tick_mem[addr],
                                bus_start_tick_coeff_mem[addr],
                                ↪ slot_vec);
eff_tk_stop  = zlc_effective_tick(bus_stop_tick_mem[addr],
                                bus_stop_tick_coeff_mem[addr],
                                ↪ slot_vec);
```

而**时间比较**(“现在该不该触发这个段?”)用辅助函数 zlc_bus_seg_start(addr, slot_vec), 它只解析段的有效起始 tick:

Verilog

```
function [TICK_WIDTH-1:0] zlc_bus_seg_start;
    zlc_bus_seg_start = zlc_effective_tick(bus_start_tick_mem[addr],
                                          bus_start_tick_coeff_mem[addr], slot_vec);
endfunction
```

它被用在 zlc_bus_start_table(启动时判断“第一段是否在 tick0 触发”: zlc_bus_seg_start(addr, slot_vec) == 0)和 zlc_bus_step(推进时判断 time_count ≥ zlc_bus_seg_start(bus_runtime_addr, slot_active))。把它做成独立函数，是为了在做时间比较时**即时 (on-the-fly)** 重算有效起始 tick，而不必额外存一份解析后的值。

为什么 DAC 值 + duration + delay 能同时扫： value_select 改的是电平 (从 slot 取 DAC 码)，仿射 tick 改的是时刻 (段随 duration/delay 平移)。这两者作用在段的不同字段上，彼此正交。所以当某个 duration 被扫描时，总线段与数字边沿锁步平移 (用同一个 `zlc_effective_tick` 公式、同一个 slot 向量)，而 value_select 仍独立扫 DAC 值——三者可以同时扫。

同时扫的算例

某 DAC edge 段: base start_tick=2000, coeff(s0)=256 (Q8 即 +1.0 tick/单位 s0) , value_select=2 (取 slot 1)。扫描点 p 的 slot 向量 s0=50、s1=900。则有效触发 tick = 2000 + (256×50)≫8 = 2000 + 50 = 2050; DAC 码 = slot1 低 10 位 = 900。下一个点 s0=80、s1=300 → tick=2080、码 = 300。段随 s0 平移 30 个 tick，同时 DAC 码从 900 变到 300——时刻与电平各自独立地被扫描。

唯一限制：斜坡端点不能是被扫描的值。也就是说 ramp 段的 start/stop_value 不能走 value_select。若需要让一个被扫描的 DAC 电平参与，请用 edge (hold) 而非 ramp 来承载它。理由是 DDA 斜坡的 delta/denom 在段应用时就一次性算定，端点若每点变动会破坏内插的整数比例。

11.6 主机到 RTL 的数据流

总线段的写入与边沿表共用 `prog_we` 写使能脉冲机制。主机侧(Zou_lab_control/neutral_atom/devices/sequencer.py 的 RuntimeBusSegment 与编译器 compile_pulse_table_scan_runtime_program) 把每段铺成一组 VIO 探针字 (probe word)，逐字段写进对应 RAM。写入路径示意：

Verilog

```
bus_value_select_mem[bus_prog_flat_addr]    ≤ bus_prog_value_select;
bus_start_tick_coeff_mem[bus_prog_flat_addr] <=
  ↪ bus_prog_start_tick_coeffs;
// ... start/stop tick、start/stop value、mode 同理逐字段写入
```

主机侧流程(详见 Zou_lab_control/neutral_atom/devices/fpga_pulse_streamer.py 的 `_prepare_tcl` 与 `_pack_signed`):

1. 编译器把每个 DAC 总线 segment 解析为 RuntimeBusSegment，字段含 start/stop tick、start/stop value、mode、value_select、start/stop_tick_coeffs。
2. 遇到表达式里的 sN (绑定到 slot N 的 DAC 框) 时，发出 value_select = N+1；普通字面值则 value_select = 0。
3. 若该段所在的 period start / delay 被扫描，仿射求解器为段算出 (base, coeffs)，把 coeffs 用 `_pack_signed` 打成 64 位 (每 slot 一个 16 位有符号 Q8)。不扫描时 coeffs 全 0，等价于普通固定 tick。
4. Vivado Tcl 会话逐段把这些字 stage 到 VIO probe，再用 `prog_we` 脉冲把每行写进分布式 RAM；同时把每路有效段数打进 `bus_counts` (28 位)。
5. 释放复位后，`zlc_bus_start_table` 用第一个扫描点的 slot 向量启动各路总线，`zlc_bus_step` 每拍按有效 tick 推进段、跑 DDA、解析 value_select。

修改清单——加一路 DAC 或扩 slot 数时

若改 BUS_COUNT：同步 bus_counts 宽度 (4→N×7 位)、zlc_bus_count_at 的 case 分支、bus_out 宽度、top 层 probe 与 create_project tcl 的 VIO IP。若改 NUM_SLOTS：COEFF_BITS、SLOT_BITS、coeff probe 宽度 (64→N×16)、value_select

的 `BUS_SEL_WIDTH` (需能编码 0..N) 随之变。任何改动都要让 Python 宽度契约测试 (`test_..._vio_widths_match_python_generator`) 重新通过, 并跑 host↔RTL 逐 tick 行为仿真核对。务必保持“运行时每路一拍只读一个段地址”这条 Artix-7 LUT 预算的核心不变量。

12

VIO probe 契约与上传流程（深入）

本章面向需要读懂、修改 RTL 与上传链路的硬件工程师。它把顶层 `zlc\_pulse\_streamer\_top\_address\_switch.v` 暴露给 Vivado VIO (Virtual Input/Output) 核的全部探针逐位讲清楚，解释为什么要把若干信号打包成宽探针，并给出从主机端 `_prepare_tcl` 写入 FPGA 的完整 `prepare/fire/差分上传` 时序。

12.1 为什么用 VIO 而不是 AXI

整个脉冲流送器跑在 `xc7a35tfgg484-2` (Artix-7 35T, 约 20.8k LUT) 上。控制面（写边沿表、扫描点表、总线段表、各种循环参数）属于低频、单次配置类操作：主机在每次实验前把整张表写下去，然后只发一个 `start` 脉冲，FPGA 就自主把序列跑完。对这种“配置一次、自由运行”的模式，使用一条完整的 AXI 总线或自定义寄存器接口都是浪费 LUT 的。

Vivado 的 `vio_0` 核恰好提供了一组可由 Tcl（通过持久化的 `VivadoPulseStreamerSession`）直接读写的 probe。`probe_out` 是 FPGA 的输入（主机 → FPGA 的控制位），`probe_in` 是 FPGA 的输出（FPGA → 主机的状态位）。这样控制面完全不占用任何数据通路逻辑，写入由 JTAG/VIO 后门完成，LUT 预算几乎只花在真正的流送数据通路上。

关键约束：bitstream 里 `NUM_SLOTS=4` 是固定的

扫描槽数 `NUM_SLOTS=4` 在综合时就写死进了 bitstream。它决定了所有打包宽探针的位宽（见下文）。改 `NUM_SLOTS` 必须重新综合、重新生成 `vio_0` IP、并同步 Python 生成器与契约测试，不能只改主机端。

12.2 顶层例化与 `vio_0` 核

顶层 `zlc\_pulse\_streamer\_top\_address\_switch.v` 例化一个 `vio_0`，配置为 30 个 `probe_out`(编号 0..29) 和 2 个 `probe_in`。这个 IP 由 `create\_project\_address\_switch.tcl` 生成，关键设置是 `CONFIG.C_NUM_PROBE_OUT 30` 加上每条探针的逐位宽度(`CONFIG.C_PROBE_OUT0_WIDTH ... C_PROBE_OUT29_WIDTH`，以及两条 `C_PROBE_IN*_WIDTH`)。

真相源(source of truth)是.v 文件,不是生成的示例。 `zlc\_pulse\_streamer\_top\_address\_switch.v` 是 Vivado 综合实际读取的顶层；Python 端的 `generate_pulse_streamer_top_example`（在 `fpga\_pulse\_streamer.py` 中）只是把同样的探针布局镜像出来，供契约测试比对。两边一旦漂移，测试就会失败（见 ?? 可用，故此处用文字描述：见“契约测试”一节）。

12.3 完整 30 条 probe_out 表

下表给出每条输出探针的编号、名字、位宽与含义。位宽决定了 Tcl 写入时该探针接受的数值范围，也决定了 vio_0 IP 的对应 C_PROBE_OUTn_WIDTH。

- 0 reset (1) 复位线。上传期间拉高保持复位，写完所有表后释放。复位高时数据通路冻结、所有写入安全。
- 1 start (1) 启动线。fire 时打一个 low-high-low 脉冲，FPGA 只把同步后的上升沿当作 start（见 fire 一节）。
- 2 prog_we (1) 边沿表写使能。每写完一行边沿的地址/tick/系数/掩码后翻转一次，把暂存内容锁进该行。
- 3 prog_addr (10) 边沿表写地址。10 位 = 1024 行边沿（这是边沿表深度）。
- 4 prog_tick (32) 该边沿的基准绝对 tick (base_tick)。32 位无符号绝对计数，默认 50 MHz / 20 ns 每 tick。
- 5 prog_mask (62) 该边沿的 62 位通道掩码。每位对应一个输出通道在该边沿之后的电平。62 位即顶层 62 个数字通道。
- 6 prog_count (11) 边沿表有效行数。11 位足以表示 0..1024。
- 7 repeat_forever (1) 若置 1，序列跑完后无限循环（重复 forever）。
- 8 loop_start_addr (10) 循环起点的边沿表地址。
- 9 loop_end_tick (32) 循环终点的基准 tick。
- 10 loop_count (32) 循环次数（非 forever 时使用）。
- 11 prog_tick_coeffs (64) **打包宽探针**：该边沿 tick 的 NUM_SLOTS 个扫描系数， $\text{NUM_SLOTS} \times \text{COEFF_WIDTH} = 4 \times 16 = 64$ 位。
- 12 scan_enable (1) 扫描总开关。1 时启用扫描点迭代。
- 13 scan_prog_we (1) 扫描点表写使能。每写完一个扫描点翻转一次。
- 14 scan_prog_addr (10) 扫描点表写地址。10 位 = 1024 个扫描点。
- 15 scan_prog_values (128) **打包宽探针**：一个扫描点的 NUM_SLOTS 个槽值， $\text{NUM_SLOTS} \times \text{TICK_WIDTH} = 4 \times 32 = 128$ 位。
- 16 scan_count (11) 扫描点总数。11 位 = 0..1024。
- 17 loop_end_coeffs (64) **打包宽探针**：循环终点 tick 的 NUM_SLOTS 个系数， $4 \times 16 = 64$ 位。
- 18 bus_prog_we (1) 总线（DAC）段表写使能。每写完一个总线段翻转一次。
- 19 bus_prog_bus (2) 目标总线索引。2 位 = 4 条 10b DAC 总线。
- 20 bus_prog_addr (6) 总线段表写地址。6 位 = 每条总线最多 64 段。
- 21 bus_prog_start_tick (32) 该段斜坡起点的基准 tick。
- 22 bus_prog_stop_tick (32) 该段斜坡终点的基准 tick。
- 23 bus_prog_start_value (10) 段起点 DAC 码（10 位 = 0..1023）。
- 24 bus_prog_stop_value (10) 段终点 DAC 码（10 位）。

25 `bus_prog_mode` (2) 段模式 (如 edge/hold/ramp 的编码)。2 位。

26 `bus_counts` (28) 四条总线各自的段数打包。28 位 = 4×7 , 每条总线 7 位计数 (0..64)。

27 `bus_prog_value_select` (3) 值来源选择。3 位: 0 = 用字面 start/stop_value; $j + 1$ = 运行时读扫描槽 j 的低 `BUS_WIDTH` 位作为 DAC 码。3 位足以编码 0..`NUM_SLOTS` (0..4)。

28 `bus_prog_start_tick_coeffs` (64) **打包宽探针**: 段起点 tick 的 `NUM_SLOTS` 个系数, $4 \times 16 = 64$ 位。

29 `bus_prog_stop_tick_coeffs` (64) **打包宽探针**: 段终点 tick 的 `NUM_SLOTS` 个系数, $4 \times 16 = 64$ 位。

两条 `probe_in` (FPGA → 主机):

`in 0 running` (1) 序列正在运行。

`in 1 done` (1) 序列运行结束。主机 `wait_done` 轮询这一位。

12.4 为什么要打包宽探针

有五条探针是**打包(packed)**的宽位向量: 11 号 `prog_tick_coeffs` (64)、15 号 `scan_prog_values` (128)、17 号 `loop_end_coeffs` (64)、28/29 号 `bus_prog*_tick_coeffs` (各 64)。它们之所以是“一条宽探针”而不是“`NUM_SLOTS` 条窄探针”, 原因如下:

- **探针条数有限, 声明成本低**。每条 VIO `probe_out` 都要在 IP 配置里单列一个 `C_PROBE_OUTn_WIDTH`, 在顶层例化里单接一根线、在契约测试里单列一项。把“每个扫描槽一份”的同类系数合并成一条宽探针, 把探针总数从随 `NUM_SLOTS` 增长压成常数条, IP 和顶层都更简洁。
- **语义是一个原子整体**。一行边沿的 4 个 tick 系数、一个扫描点的 4 个槽值, 本来就要一起写、一起锁进同一行分布式 RAM。打包成一个字, 一次 Tcl 写入 + 一次 `prog_we` 翻转即可, 无需分多次写多条探针再担心它们的写入是否落在同一拍。
- **位宽来自 `NUM_SLOTS` 的乘积, 固定可算**。`COEFF_WIDTH=16`、`TICK_WIDTH=32`, 于是系数包 = $4 \times 16 = 64$, 槽值包 = $4 \times 32 = 128$ 。这些常数直接出现在 IP 的 width 配置里。

12.4.1 打包位序: `_pack_signed`

主机端用 `_pack_signed` 把 N 个宽 W 的**有符号值**按**小端 (little-endian)** 打进一个无符号字, 用于系数探针与扫描值探针。槽 j 占据字的第 $[j \cdot W .. j \cdot W + W - 1]$ 位, 槽 0 在最低位。负值按二进制补码截断到 W 位后放入对应位段。

小端槽布局举例 (系数包, $W=16$, $N=4$)

设四个系数 $c_0 = +1$, $c_1 = -1$, $c_2 = 0$, $c_3 = +2$ 。各自补码 16 位: $c_0 = 0x0001$, $c_1 = 0xFFFF$, $c_2 = 0x0000$, $c_3 = 0x0002$ 。小端拼成 64 位字 (槽 0 在低位):

`word = 0x0002_0000_FFFF_0001`

读回时 RTL 把 64 位按 16 位切片、第 j 段当有符号数即得 c_j 。这与 RTL 内 `prog_tick_coeffs[j*16 +: 16]` 的切片完全对应。

RTL 侧拿到这个打包字后, 逐槽切片参与有效 tick 计算:

text

```
effective_tick = base_tick + ( sum_j coeff_j * slot_j ) >> FRAC
```


其中 `base_tick` 是该边沿/段的基准 tick (probe 4 或 21/22), `coeff_j` 来自打包系数探针的第 j 段, `slot_j` 是当前扫描点该槽的值 (来自 `scan_prog_values` 写入的扫描点表), `FRAC` 是定点小数位数。这一个表达式同时被边沿 tick、循环终点 tick、总线段起止 tick 复用——这正是不做 spec 里 per-channel run-length 重写、而沿用“仿射流式扫描”的核心：所有需要随扫描平移的时间点都走同一个 `zlc_effective_tick`, 省 LUT。

12.4.2 分布式 RAM 单读口的影响

边沿表、扫描点表、总线段表都映射到 Artix-7 的**分布式 RAM (distributed RAM, 用 LUT 做的小 RAM)**。分布式 RAM 是**单读口**的：一拍只能从一个地址读出一份数据。这对设计有两点直接后果：

- **宽探针正好契合单口读**。把一行的所有槽系数打包成一个宽字存进一个地址，一次读地址就拿到整行 4 个系数，不需要在同一拍读多个地址。若拆成 4 条窄 RAM，要么 4 个单口 RAM、要么多周期读，都更费资源或更慢。
- **扫描边界处要显式传 slot 向量、避免 NBA 时序坑**。在扫描点切换的边界，若直接在组合逻辑里引用还未稳定的 `slot_active` (非阻塞赋值尚未更新)，会读到上一拍的值。因此总线引擎的 `apply/start_table` 任务被改为**显式传入 slot_vec 参数**，由调用点保证传进来的是当前正确的槽向量，绕开单读口 + NBA 更新时序的竞争。

12.5 契约测试：.v 与 Python 生成器对齐

有一个 Python 契约测试断言**生成的 VIO 宽度在签入的顶层.v 与 Python 生成器之间逐条一致**。机制是：

1. 真相源是 `zlc\_pulse\_streamer\_top\_address\_switch.v` (综合实际用的顶层) 与 `create\_project\_address\_switch.tcl` (生成 `vio_0` 的 30 探针宽度)。
2. `generate_pulse_streamer_top_example` (`fpga\_pulse\_streamer.py`) 按同样的 NUM_SLOTS 与位宽规则**重新生成**一份等价的顶层例子。
3. 测试比对两者的每条 `probe_out` 宽度 (0..29 共 30 条) 与 2 条 `probe_in`，全部相等才通过。

这层契约的价值在于：当有人改了 RTL (比如新增 `probe_out`、或改某条探针位宽)，却忘了同步 Python 生成器/Tcl/IP 配置，测试立刻红，防止 RTL ↔ top ↔ tcl ↔ 主机生成器之间**悄悄漂移**。注意测试只保证**位宽契约**对齐，不替代硬件行为仿真——后者靠 host↔RTL 的逐 tick 行为仿真覆盖。

改一条探针要同步的所有地方

若要新增/加宽一条探针,必须同时改:(1) `zlc\_pulse\_streamer\_top\_address\_switch.v` 的端口与 `vio_0` 例化;(2) `create\_project\_address\_switch.tcl` 的 `C_NUM_PROBE_OUT` 与 逐条 `C_PROBE_OUTn_WIDTH`;(3) `generate_pulse_streamer_top_example` 的镜像生成;(4) 主机 `_prepare_tcl` 的 staging 顺序;(5) 契约测试的期望宽度。漏一处,契约测试或综合会报错。

12.6 上传流程：prepare (_prepare_tcl)

主机通过 `_prepare_tcl` 把整个 `RuntimeSequenceProgram` 写进 FPGA。顺序经过精心设计，全程在复位保护下完成：

1. **拉高 reset 保持**。复位高时数据通路冻结，所有写入安全，FPGA 不会在写到一半时误启动。

2. **暂存全局/循环参数**: 写 `repeat_forever`、`loop_start_addr` / `loop_end_tick` / `loop_count` / `loop_end_coeffs`、`scan_enable` / `scan_count`、`bus_counts`、`prog_count`。这些是不进逐行 RAM 的标量寄存器，直接 stage 即可。
3. **逐边沿写边沿表**: 对每一行边沿，先 stage `prog_addr` / `prog_tick` / `prog_tick_coeffs` / `prog_mask`，然后翻转 `prog_we` (toggle) 把这一行锁进该地址。系数用 `_pack_signed` 打包成 64 位字。
4. **逐扫描点写扫描点表**: 对每个扫描点，stage `scan_prog_addr` / `scan_prog_values` (128 位打包槽值)，然后翻转 `scan_prog_we`。
5. **逐总线段写总线段表**: 对每个 DAC 段，stage `bus_prog_bus` / `bus_prog_addr` / `bus_prog_start_tick` / `bus_prog_stop_tick` / `bus_prog_start_tick_coeffs` / `bus_prog_stop_tick_coeffs` / `bus_prog_start_value` / `bus_prog_stop_value` / `bus_prog_mode` / `bus_prog_value_select`，然后翻转 `bus_prog_we`。
6. **释放 reset**。表全部就位后拉低复位，FPGA 进入待命，等待 start。

每张表都靠“先 stage 各字段、再翻转对应 we”的模式写入：翻转 we 在 RAM 写口产生一个写脉冲，把暂存字段锁进 `*_addr` 指向的那一行。地址在每行之间递增即可顺序填满整张表。

12.7 fire: 只认同步上升沿

`fire` 在 `start` 探针上打一个 **low-high-low** 脉冲。FPGA 内部对 `start` 做跨时钟域同步后**只把同步后的上升沿当作启动信号**，因此：

- `start` 在 `prepare` 阶段保持低，避免上传途中误触发；
- 一次 `fire` 只产生一个上升沿 → 只启动一次，电平宽度无关紧要 (`low-high-low` 保证之后回低，下一次 `fire` 能再产生新的上升沿)；
- 用边沿而非电平触发，杜绝 VIO 写入抖动或保持高位造成的重复启动。

启动后主机 `wait_done` 轮询 `probe_in` 的 `done` (必要时配合 `running`) 判断序列是否跑完。

12.8 差分上传 (differential upload)

完整 `prepare` 要逐行写满边沿表 (最多 1024 行)、扫描点表 (最多 1024 点)、总线段表，VIO/JTAG 写入并不快。对“只改了少量参数”的连续实验，全量重写是浪费。因此实现了**差分上传**：

- **首次 prepare 仍全量写**，建立 FPGA 内 RAM 的完整内容，并在主机侧记录“上次写了什么”的影子 (shadow)。
- **之后只重写变化的行**：仅改变的边沿行、改变的扫描点，加上**影子关键行 (shadow-critical rows)**——即第 0 行、循环起点行、最后一行——会被重写。这些关键行即使内容没变也要保证写对，因为它们界定了序列的起止与循环边界，是流送引擎正确性的锚点。
- 标量参数 (`loop/scan/bus_counts/prog_count` 等) 按需 stage，复位包络同 `prepare`。

这样把“换一个扫描数组”或“挪一两个边沿”的上传时间从 $O(\text{表全长})$ 降到 $O(\text{改动行数} + \text{常数个关键行})$ ，同时用强制重写关键行的策略守住正确性。

差分上传为什么要强制写关键行

单读口分布式 RAM + 流送引擎在 row 0 / loop start / last 这几行上做边界判断 (序列开始、循环回跳、序列结束)。即便主机的影子认为它们“没变”，也要重写以排除任何影子失配 (例如上一次会话遗留、或位打包细节变动) 导致的边界错位。代价只有常数几行，换来边界绝对正确，是值得的保守策略。

13

JTAG-to-AXI 边沿表加载器（当前架构）

13.1 为什么放弃 per-channel run-length

上一轮曾按 spec 做单通道 run-length 引擎（每数字通道一个硬件 player，走自己的转换列表）。在 35T 的**单个 BRAM 读口**下，每个 repeat/scan 边界都要重新装载 62 个通道 player，实测约 $23.6\ \mu\text{s}$ ——短重复段根本无法无缝。因此**放弃**该方案，回到已逐 tick 验证的**全局仿射 edge-table 引擎** `zlc_pulse_streamer.v`。它只有一个全局边沿指针，所以：

- repeat = 单周期硬件 loop 回卷（影子寄存器重载，`zlc_pulse_streamer.v` 第 470–475 行）
⇒ **0 gap**;
- 扫描点切换 = 单周期换 slot 向量（第 477–493 行）⇒ **0 gap**;
- LUT 更省——一个比较器 + 一条 `zlc_effective_tick` MAC，而非 62 个 player。

这与商用脉冲流发生器一致：Swabian Pulse Streamer 是全局 RLE 指令流 + 硬件 loop，SpinCore PulseBlaster 是全局指令流 + LOOP opcode——都不用逐通道硬件 player。

13.2 控制路径：片上加载器（替代 VIO）

引擎**一个字节都没改**，仍是 `zlc_pulse_streamer.v` (host↔RTL 逐 tick 验证过的那个)。改变的只是程序怎么进引擎：用 JTAG-to-AXI 把程序映像写进 BRAM，再由片上加载器 `zlc_axi_program_loader.v` 把映像拷进引擎的 `prog_*/scan_prog_*/bus_prog_*` 写口（引擎自己的写契约：复位保持时翻转 `prog_we` 提交一行，写入期间影子寄存器从 `prog_count/loop_start_addr` 锁存），随后松复位、发 start。因为引擎未变，无缝与 tick 精确原样保留——这也是不重写引擎、只换送数方式的关键。

13.3 各文件

- `devices/edgetable_image.py`: 把 `RuntimeSequenceProgram` 打包成 32 位 BRAM 映像——偏移 0 的 CTRL 块 (magic、COMMAND/STATUS 邮箱、prog_count、scan_count、loop 字段、bus_counts、slot_count)，随后 EDGE 表 (tick + N-slot 系数 + 62 位 mask, 5 字/边沿)、SCAN 点表 (`NUM_SLOTS` 字/点)、按总线的 BUS 段表 (仿射起止 tick + 系数 + 值/模式/`value_select`, 7 字/段)。只发非空行（稀疏），典型程序几百字。`unpack_program` 是加

载器走表的解码器; `pack(p)` 再 `unpack` 等于 `p` 是契约。

- `zlc_axi_program_loader.v`: 顺序 FSM, 非时序关键 (prepare 期间跑)。轮询 COMMAND (上升沿检测 LOAD/FIRE/SAFE/RESET), LOAD 时按 `unpack_program` 的顺序走映像、驱动引擎 toggle 写口 (每行数据保持稳定、每次读 settle 以容忍读延迟 1 或 2), FIRE 时松复位 + 发 start, 并把 STATUS (LOADED/RUNNING/DONE/ERROR) 写回 BRAM 供主机轮询。`repeat_forever` 永不置 DONE, 故 FSM 持续轮询命令 (永久运行中 SAFE/RESET/LOAD 仍有效)。
- `devices/edgetable_loader_model.py`: 仓库无 Verilog 仿真器, 故把加载器 FSM 在 Python 里逐周期镜像, 校验其捕获的引擎写能重建解码后的映像——这是新时序器走表顺序/地址/字段切片的上机前证据 (含 200 边沿压测 + 无 scan/无 bus 边界)。
- `zlc_pulse_streamer_loader_top.v`: `jtag_axi_0` → `axi_bram_ctrl_0` → `blk_mem_gen_0` (真双口): A 口 = AXI 上传 + 邮箱, B 口 = 加载器。加载器 → 引擎; 引擎 out (62) + bus_out (40, DAC 现已接上) → 原样 62 脚 address_switch 板图。
- 构建 `create_project_loader.tcl` (严格 32768 BRAM 深度 + 回读守护, 短项目名 l 是 Vivado 路径长度修复); `program_fpga_loader.tcl` 让 `jtag_axi` 核作为 `hw_axi` 可见。
- `devices/axi_session.py` (`VivadoAxiStreamerSession`) : 一个常驻 vivado -mode tcl; `prepare` 打包 + 上传映像再发 LOAD (等 STATUS_LOADED), `fire` 发 FIRE, `wait_done` 轮询 STATUS_DONE (`repeat_forever` 用 RUNNING 当完成), `safe_state` 发 SAFE。COMMAND 邮箱发命令前先写 0 以保证干净上升沿。Tcl 执行器可注入, 整条 pack → 上传 → 加载器模型 → LOAD → fire 流程无需 Vivado 即被单测覆盖。

13.4 当前容量与待办

62 数字 + 4×10 位 DAC; 20 ns tick; `NUM_SLOTS`=4 个仿射槽 (delay/duration/DAC 任意组合都可同时扫); 引擎 LUTRAM 表 ≤ 1024 边沿 + 1024 扫描点 (映像 BRAM 能放更多)。on_pulse → 首个输出 = 1 周期 (20 ns); <100 ms 预算只是上传, 已加载程序的 re-fire 只是几次控制写。**v-next**: 更多边沿/点 (引擎表搬 BRAM + 1 边沿预取流水), 以及冷加载 <100 ms 的 burst / AXI4-full 上传; 当前单字上传正确但几百字冷载约亚秒, 之后无限次快速 re-fire。

13.5 构建 / 运行

`fpga/build_and_program.bat` (无参数双击) 构建 + 烧录加载器位流到 `fpga/build/l`; 随后 `fpga/run_server.bat` 以默认后端 `jtag-axi` 连接并提供 RPyC 服务。GUI/notebook 的 Run 照常走 `prepare` → LOAD → `fire`, 引擎从 LUTRAM 无缝播放。

14

Architecture-D: BRAM 表 + 预取 (2048 edges + 4096 points)

14.1 为什么需要 D

加载器路径把整张程序拷进引擎的 LUTRAM 表，所以容量受分布式 RAM (35T 约 400 Kib $\approx$ 6400 个 LUTRAM-LUT) 限制：1024 edges + 1024 points 已接近上限，**2048 edges + 2048 points 纯 LUTRAM 放不下** (约需 11.5k LUTRAM-LUT)。要达到用户要求的 2048+/2048+ 且每项资源 $\leq 75\%$ ，**edge 表和 scan 表必须搬进 block RAM**。模拟总线段表**留在 LUTRAM**——因为总线/斜坡引擎**每 tick 组合读**这些表，搬进 BRAM 会破坏它。这就是 “D” 的取舍 (经对抗复核确认优于把总线也搬 BRAM)。

14.2 depth-1 预取与最小边沿间距

BRAM 是**同步读** (1-2 周期延迟)，而引擎要**同周期**比较 `time_count = effective_tick` 才能无缝触发。`zlc_pulse_streamer_d.v` 用 **depth-1 预取**解决：当前边沿 `cur` 是一个**寄存器**——边沿 0 来自 `first` 影子 (arm 时预读，零延迟)，其余边沿在 `edge_index` 前进时从 BRAM 读入 `pre_*`，约 `RD_SETTLE` 周期后有效。因为一条边沿一触发就立刻发起下一条的读，而下一条至少在**最小边沿间距**之外，`pre_*` 总能在它的 effective tick 之前就绪——所以触发仍是同周期、无缝。四个无缝重载点 (start / loop 回卷 / scan 推进 / repeat) 本就从影子寄存器重载，零 BRAM 延迟，和已验证的组合引擎一致。

代价：最小边沿间距 = `RD_SETTLE+1 = 3 tick = 60 ns` (50 MHz)。这不是核心需求 (核心需求是 repeat/scan 之间无缝，而非两条边沿能多近)，主机编译/上传时**强制**该约束并清晰报错 (`axi_session` 调 `min_edge_spacing`, < 3 则拒绝)。

无仿真器下的证明：`devices/edgetable_engine_model.py` 的 `prefetch_d1_play` 在间距 ≥ 3 时与已验证组合引擎 `reference_play` **逐 tick 字节相同**、更密时正确 STALL (见 `test_edgetable_d1_prefetch_matches_reference_with_min_spacing` 与三方暴力 ground-truth 交叉验证)。这是把 “算法证明对” 落到这块 RTL 的依据。

14.3 容量与参数化 (随 XDC + FPGA 派生)

`devices/edgetable_image.py` 的 `solve_capacity(part, channel_count)` 是**单一真相源**：按 FPGA 部件的 RAMB36/LUT/FF/DSP 预算 + XDC 通道数，派生 `max_edges /`

`max_scan_points` / 地址位宽 / BRAM 深度, 并**校验每项资源 $\leq 75\%$** 。换 XDC 或换 FPGA 只改这一处输入, RTL localparams、create-project tcl、主机三者同源派生。

- **xc7a35t (62 通道)**: 2048 edges + 4096 scan points + 512 流式窗口, 约 31/50 RAMB36 = 62%; LUT $\sim 23\%$ 、FF $\sim 7\%$ 、DSP $\leq 9\%$ 。**全部 $\leq 75\%$** (比要求的 2048 $\times$ 2048 还多)。
- 换大板 (xc7a100t/200t) 自动放宽到同等或更大容量, 占用比例更低; 换窄 XDC (如 16 通道) 也自动适配——都不重写。

14.4 各文件

- `zlc_pulse_streamer_d.v`: D 引擎 (depth-1 预取 + 影子; 总线 LUTRAM + 总线引擎原样复用 `zlc_effective_tick/ramp/value_select`)。edge/scan 经外部 BRAM 读口进来。
- `zlc_pulse_streamer_d_top.v`: 复用已验证的 `axi_bram_ctrl` (它处理 AXI 握手), 只加一个**简单组合地址译码**把其 BRAM 端口分到 edge BRAM (非对称 32 写/256 读)、scan BRAM (32/128)、bus-image BRAM、控制寄存器组; bus 小加载器把 bus 镜像写进引擎 LUTRAM。62 脚板图 + DAC (接 `bus_out`) 原样。
- `create_project_d.tcl`: 建 3 个 BRAM IP (防御式 `zlc_try+dump`, 版本无关属性名上机收敛) + jtag_axi + axi_bram_ctrl, 综合到位流 (项目 `fpga/build/d`)。
- `devices/edgetable_image.py`: `pack_program_d/unpack_program_d` (D 的 AXI 写镜像: edge 8 字/256 位行、scan `NUM_SLOTS` 字、bus 7 字; 区基址与顶层 localparam 一致)。
- `devices/axi_session.py`: `VivadoAxiStreamerSession(variant="d")`——打包 D 镜像、强制最小边沿间距、写 BRAM、COMMAND/STATUS (LOAD $\rightarrow$ FIRE $\rightarrow$ 轮询 DONE), 与加载器路径共用 Tcl 管道。

14.5 状态与 bring-up

引擎算法(depth-1 模型 = 参考引擎)、容量求解、主机镜像往返、结构契约都已 Python 测试通过。**多-BRAM AXI 集成本身是盲写** (仓库无 Verilog 仿真器), 需在板上 bring-up: 先用 `create_project_d.tcl` 综合 (grep 日志的 ZLC IPDUMP/ZLC TRY-FAIL 收敛版本相关 IP 属性), 再上板。这是本项目一贯的 RTL 验证现实——上机前能证明的都已证明。